

Chapter 4. **Robot Arm Kinematics**

Contents

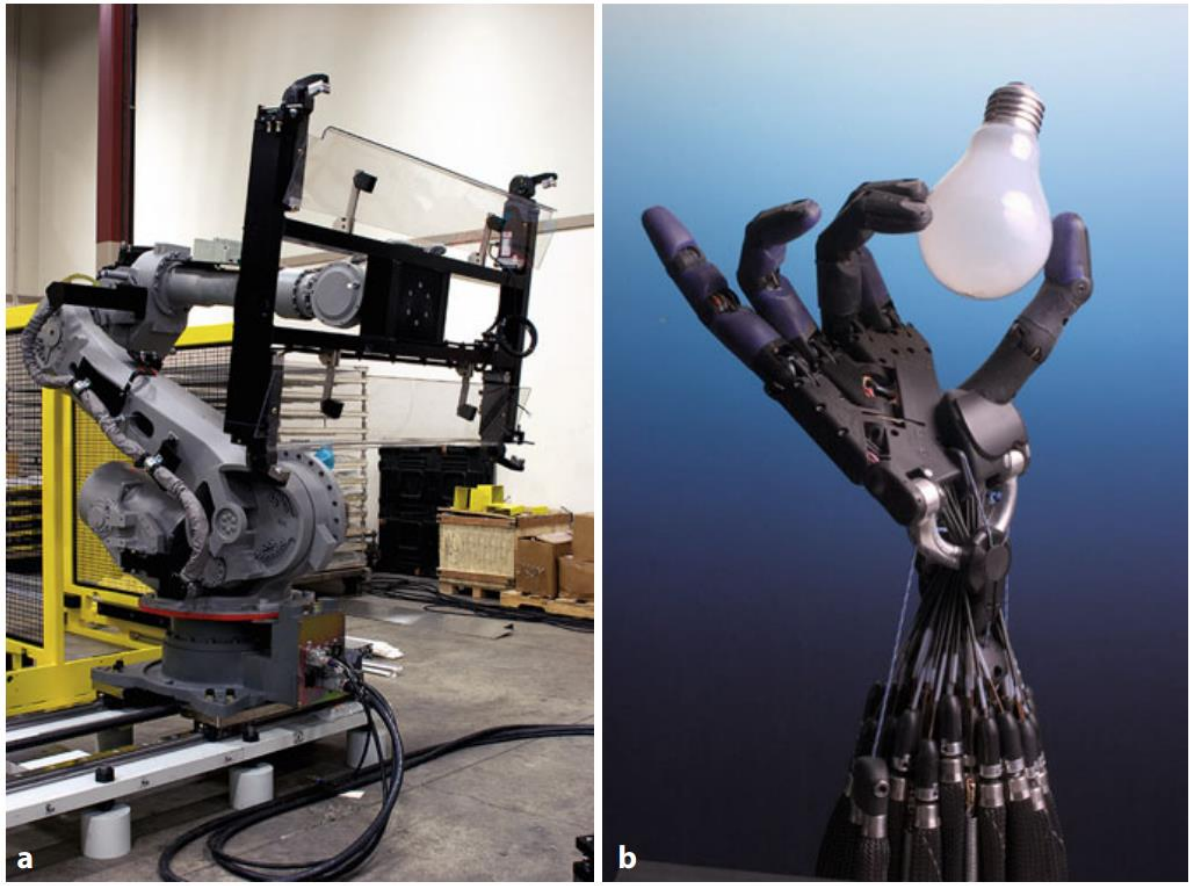
5.1 Forward Kinematics

5.2 Inverse Kinematics

5.3 Trajectories

5.4 Applications

- A robot manipulator arm comprises mechanical links and computer-controllable joints.
- The base of the robot is generally fixed and motion of the joints changes the pose of the tool or end effector in space in order to perform useful work.



DRAFT

- This chapter is concerned with kinematics – the branch of mechanics that studies the motion of a body, or a system of bodies, without considering mass or force.
- The pose of an end effector is a function of the state of each of the robot's joints and this leads to two important kinematic problems.
- Forward kinematics is about determining the pose of the end effector from the joint configuration.
- Inverse kinematics is about determining the joint configuration given the end-effector pose.

DRAFT

5.1 Forward Kinematics

- The robots shown in the below figures are serial-link manipulators and comprise a series of rigid links connected by actuated joints.
- Each joint controls the relative pose of the two links it connects, and the pose of the end effector is a function of all the joints.
- A series of links and joints is a kinematic chain or a rigid-body chain.





DRAFT

- In robotics the most common type of joint is revolute, where one link rotates relative to another, about an axis fixed to both.
- Less common is a prismatic joint, also known as a sliding or telescopic joint, where one link slides relative to another along an axis fixed to both.
- Many other types of joints are possible such as screw joints (1 DoF), cylindrical joints (2 DoF, one revolute and one prismatic) or spherical joints (3 DoF).





DRAFT

- We are interested in understanding the relationship between the joints, which we control, and the pose of the end effector which does the work.
- This is the forward kinematics, a mapping from joint coordinates, or robot configuration, to end-effector pose.
- In each section we treat the simpler 2D case first, then the 3D case.

DRAFT

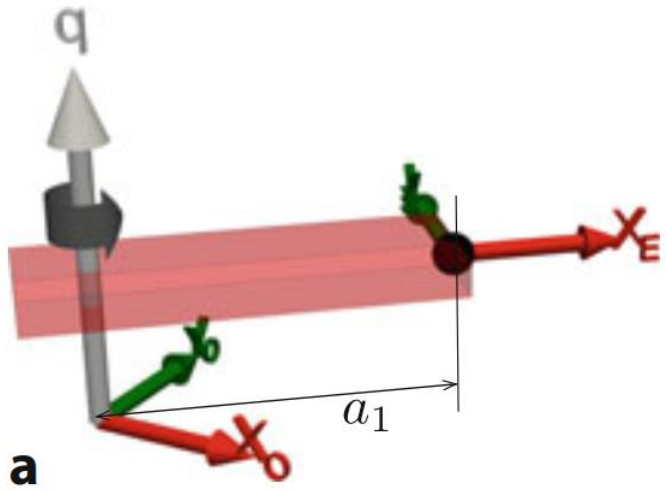
5.1.1 Forward Kinematics from a Pose Graph

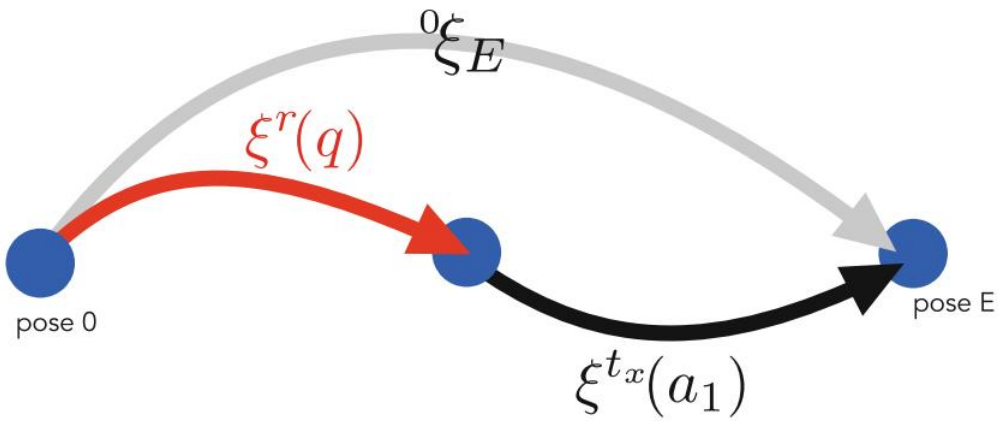
- In this section we apply our knowledge of pose graphs to solve the forward kinematics problem for robots that move in 2D and 3D.

DRAFT

4.1.1.1 2-Dimensional (Planar) Robotic Arms

- We will start simply with a single kinematic chain operating in the plane.
- Consider the very simple example which comprises one revolute joint and one link.
- The pose graph depicts the composition of two relative poses.





DRAFT

- Demonstrate an elementary transformation sequence (ETS).

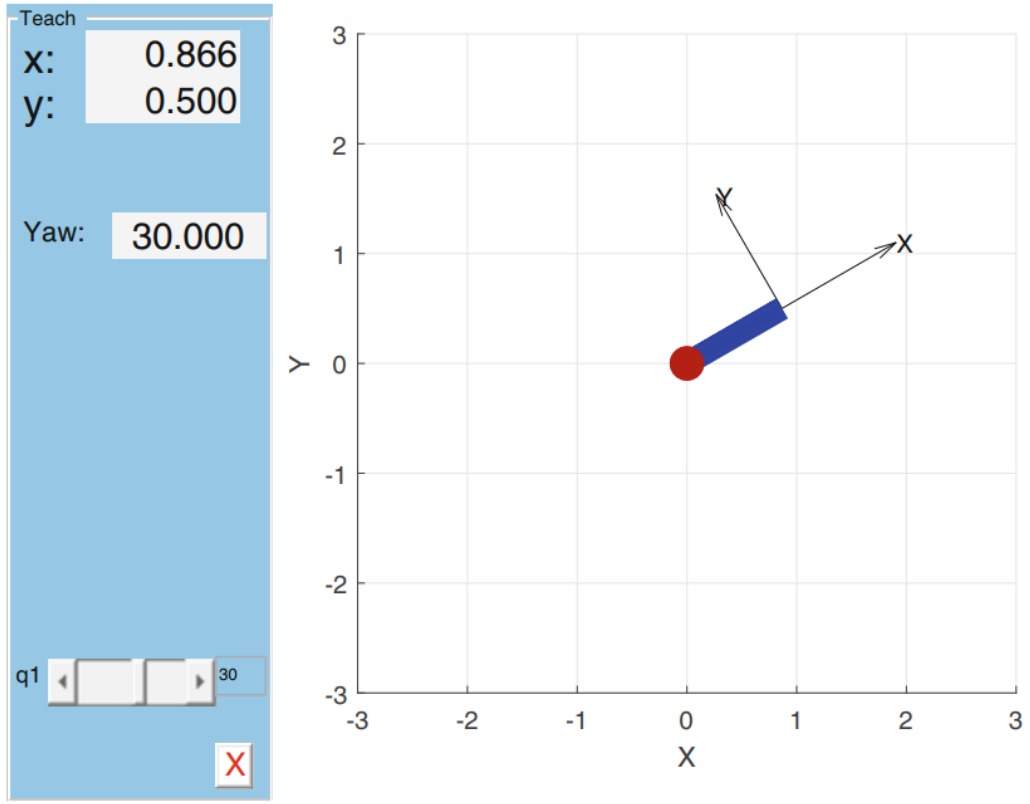
```
>> a1 = 1;
>> e = ETS2.Rz("q1")*ETS2.Tx(a1)
e =
Rz(q1)Tx(1)
```

- For a particular value of the joint variable, say $q = \frac{\pi}{6}$ radians, the end-effector pose is

```
>> e.fkine(pi/6)
ans =
    0.8660   -0.5000    0.8660
    0.5000    0.8660    0.5000
         0         0    1.0000
```

- An easy and intuitive way to understand how this simple robot behaves is by

```
>> e.teach;
```

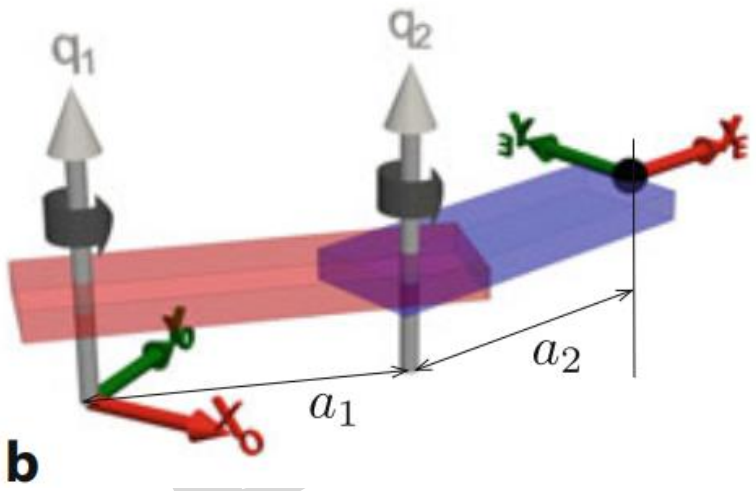


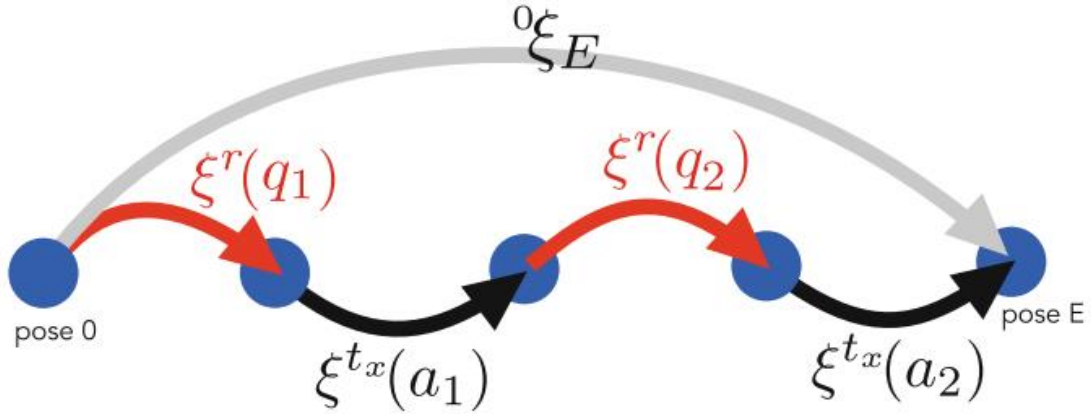
DRAFT

- Consider now the robot shown in the below figure which has two revolute joints q_1 and q_2.
- From the pose graph, the pose of the end effector is

$$^0\boldsymbol{\xi}_E = \boldsymbol{\xi}^r(q_1) \oplus \boldsymbol{\xi}^{t_x}(a_1) \oplus \boldsymbol{\xi}^r(q_2) \oplus \boldsymbol{\xi}^{t_x}(a_2)$$

```
>> a1 = 1; a2 = 1;
>> e = ETS2.Rz("q1")*ETS2.Tx(a1)*ETS2.Rz("q2")*ETS2.Tx(a2)
e =
Rz(q1)Tx(1)Rz(q2)Tx(1)
```





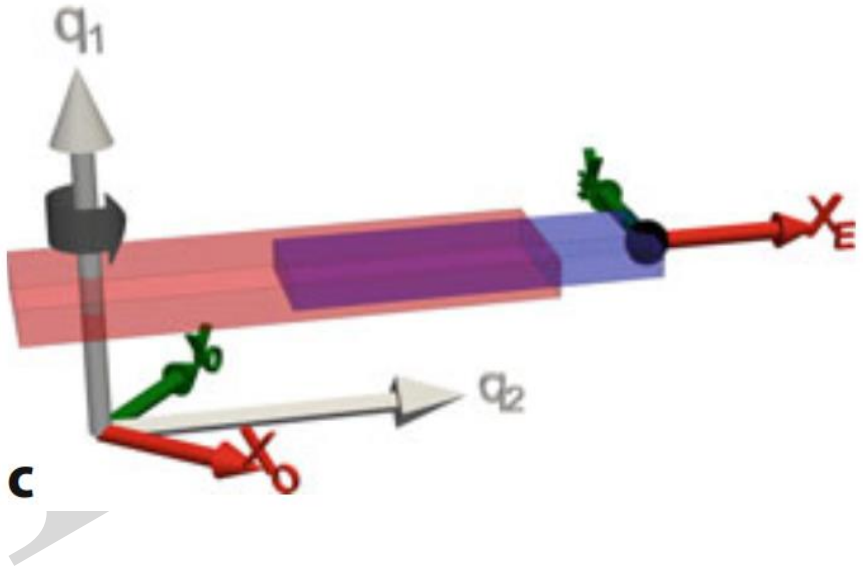
- We describe the configuration of a robot manipulator with N joints by a vector of generalized coordinates where $\boldsymbol{q}_j \in \boldsymbol{S}^1$ is an angle for a revolute joint or $\boldsymbol{q}_j \in \mathbb{R}$ is a length for a prismatic joint.
- We refer to $\boldsymbol{q}$ as the joint configuration or joint coordinates and for an all-revolute robot as the joint angles.
- This robot has 2 degrees of freedom and its configuration space is $\mathcal{C} \in S^1 \times S^1$ and $q \in \mathcal{C}$. This is sufficient to reach points in the task space $\mathcal{T} \subset \mathbb{R}^2$ since $\dim \mathcal{C} = \dim \mathcal{T}$.
- However if our task space includes orientation $\mathcal{T} \subset \mathbb{R}^2 \times S^1$ then it is underactuated since $\dim \mathcal{C} < \dim \mathcal{T}$ and the robot can access only a subset of the task space, for example, positions but not orientation.

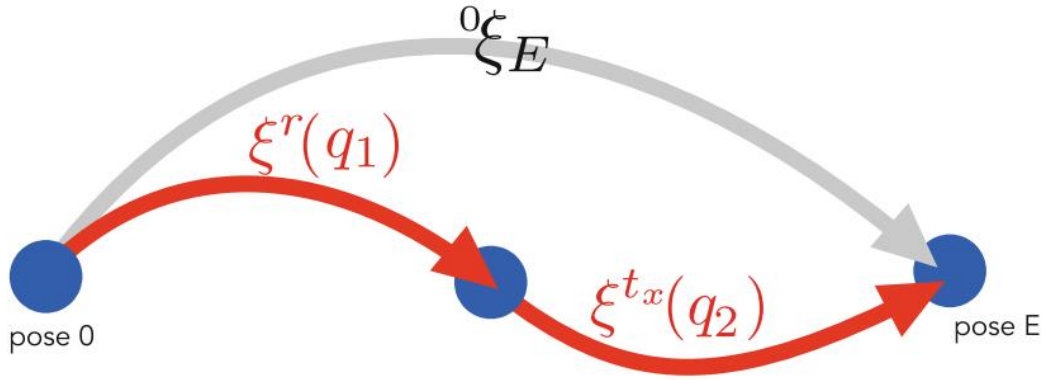
DRAFT

- We will finish up with the robot shown in the below figure which includes a prismatic joint and is commonly called a polar-coordinate robot arm.
- The end-effector pose is

$$ {}^{0}\boldsymbol{\xi}_{E} = \boldsymbol{\xi}^{r}(q_1) \oplus \boldsymbol{\xi}^{t_x}(q_2) $$

```
>> e = ETS2.Rz("q1")*ETS2.Tx("q2",qlim=[1 2])
e =
Rz(q1)Tx(q2)
>> e.structure
ans =
    'RP'
```





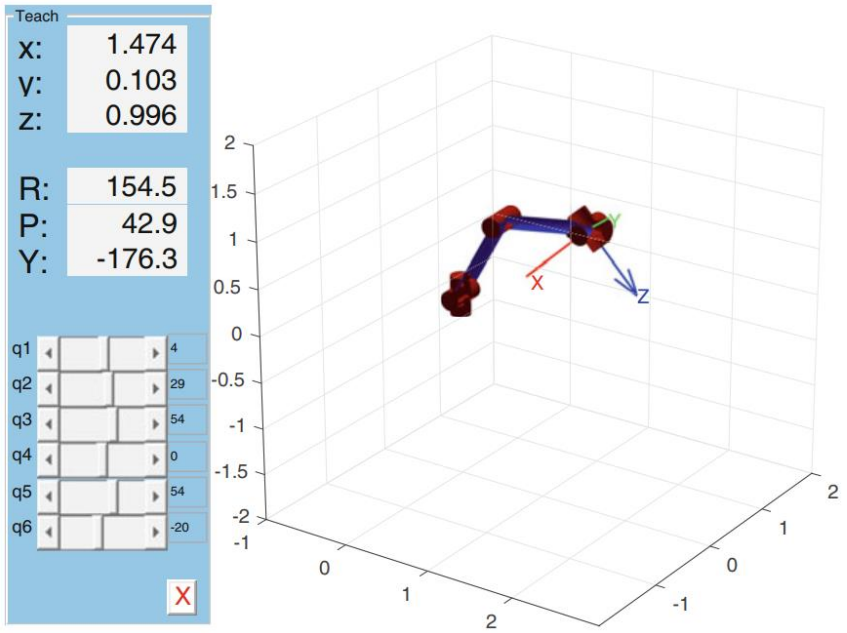
- A serial-link manipulator with N joints (numbered from 1 to N) has $N + 1$ links (numbered from 0 to N).
- Joint j connects link $j - 1$ to link j and moves them relative to each other.
- Link ℓ connects joint ℓ to joint $\ell + 1$.
- Link 0 is the base of the robot, typically fixed, and link N, is the last link of the robot and carries the end effector or tool.

DRAFT

5.1.1.2 3-Dimensional Robotic Arms

- Most real-world robot manipulators have a task space $\mathcal{T} \subset \mathbb{R}^3 \times S^3$ which allows arbitrary position and orientation of the end effector within its 3D working envelope or workspace.
- This requires a robot with a configuration space $\dim \mathcal{C} > \dim \mathcal{T}$ which can be achieved by a robot with six or more joints.

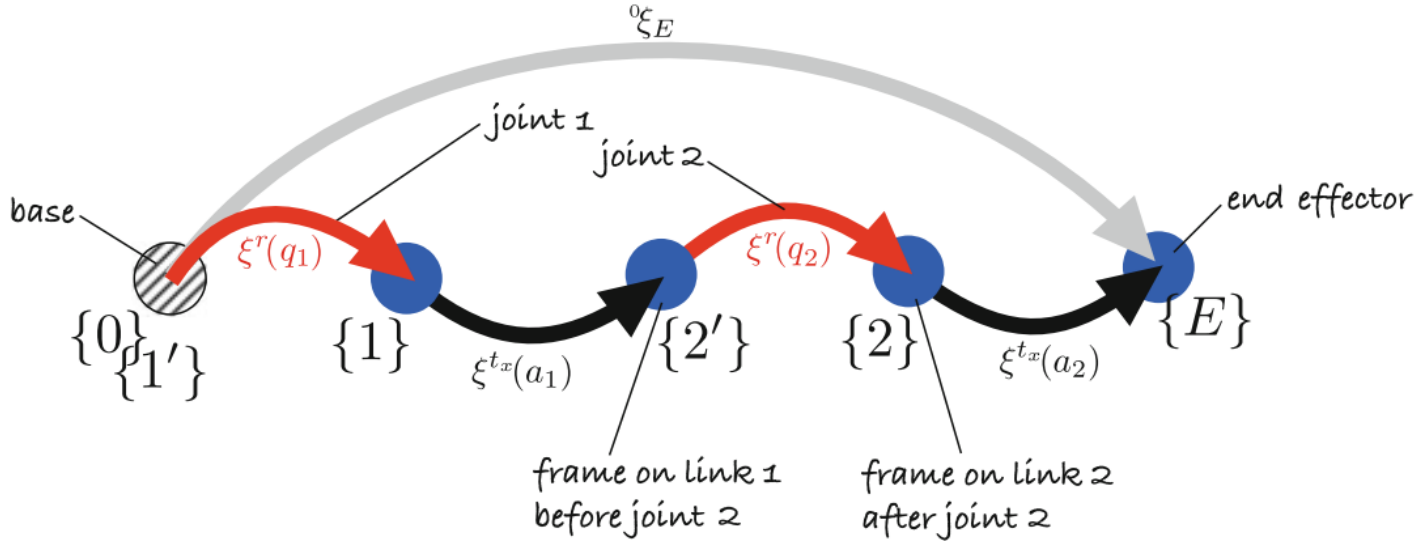
```
>> a1 = 1; a2 = 1;
>> e = ETS3.Rz("q1")*ETS3.Ry("q2")* ...
>>    ETS3.Tz(a1)*ETS3.Ry("q3")*ETS3.Tz(a2)* ...
>>    ETS3.Rz("q4")*ETS3.Ry("q5")*ETS3.Rz("q6");
```



DRAFT

5.1.2 Forward Kinematics as a Chain of Robot Links

- The below figure shows the relationship between joints and link frames.
- The frame for link ℓ is indicated as $\{\ell\}$ and its pose is a function of the joint coordinates $\{q_j, j = 1, \ldots, \ell\}$.
- We say that link ℓ is the parent of link $\ell + 1$ and the child of link $\ell - 1$.
- The fixed base is considered to be link 0 and has no parent, while the end effector has no child.



DRAFT

- The pose of the end effector, the forward kinematics is

$$ {}^{0}\boldsymbol{\xi}_N = {}^{0}\boldsymbol{\xi}_1(q_1) \oplus {}^{1}\boldsymbol{\xi}_2(q_2) \oplus \cdots \oplus {}^{N-1}\boldsymbol{\xi}_N(q_N) $$

where ${}^{j}\xi_{j+1}(q_{j+1})$ is the pose of link frame $\{j+1\}$ with respect to link frame $\{j\}$.

- We can write this in functional form as

$$ {}^{0}\boldsymbol{\xi}_N = \mathcal{K}(\boldsymbol{q}) $$

where $\mathcal{K}(\cdot)$ is specific to the robot and incorporates the joint and link parameters.

DRAFT

5.1.2.1 Robots as Rigid Body Trees

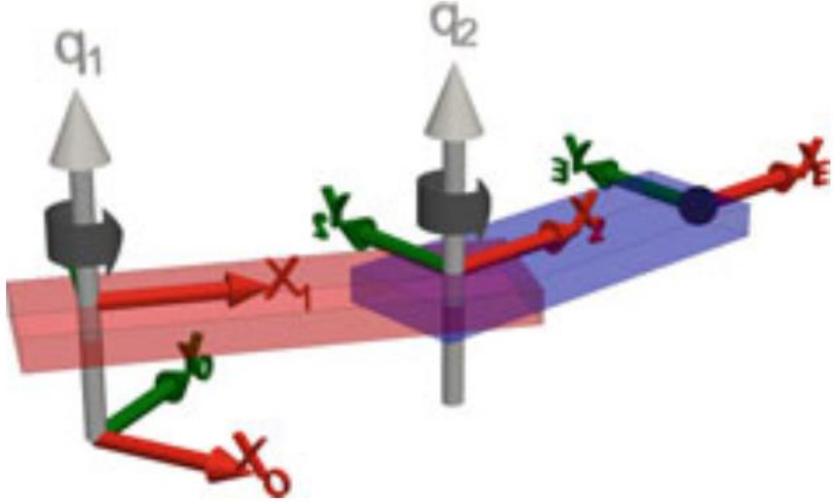
- We will leverage dedicated objects and functions that encapsulate the concepts of links, joints, and robots.
- There are three objects that are essential to describing robots:

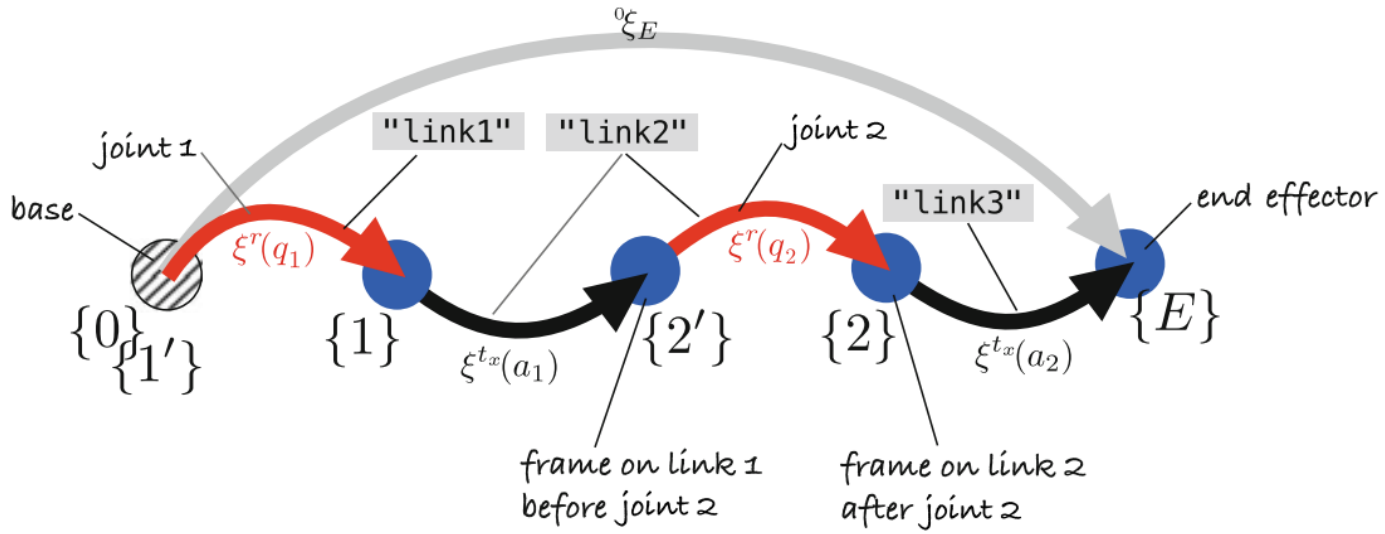
- A rigidBodyTree represents the robot structure.
- A rigid body tree is made up of rigidBody objects. These are the links of the robot and have properties such as mass, inertia, and collision geometries.
- Each link is connected to its parent with a rigidBodyJoint.

DRAFT

5.1.2.2 2-Dimensional (Planar) Case

- We consider again the robot shown in the below figure.
- The pose of the link 1 frame shown in red depends only on q_1, while the pose of the link 2 frame shown in blue depends on q_1 and q_2, as well as the length of link 1.





DRAFT

- We can describe the robot by first creating the relevant link and joint objects, each with a unique name

```
>> link1 = rigidBody("link1");
>> link1.Joint = rigidBodyJoint("joint1","revolute");
>> link2 = rigidBody("link2");
>> link2.Joint = rigidBodyJoint("joint2","revolute");
>> link2.Joint.setFixedTransform(se3([a1 0 0],"trvec"));
>> link3 = rigidBody("link3");
>> link3.Joint.setFixedTransform(se3([a2 0 0],"trvec"));
```

- We can now create a robot object and add these links to it.

```
>> myRobot = rigidBodyTree(DataFormat="row");
>> myRobot.addBody(link1,myRobot.BaseName);
>> myRobot.addBody(link2,link1.Name);
>> myRobot.addBody(link3,link2.Name);
```

DRAFT

5.1.2.3 3-Dimensional Case

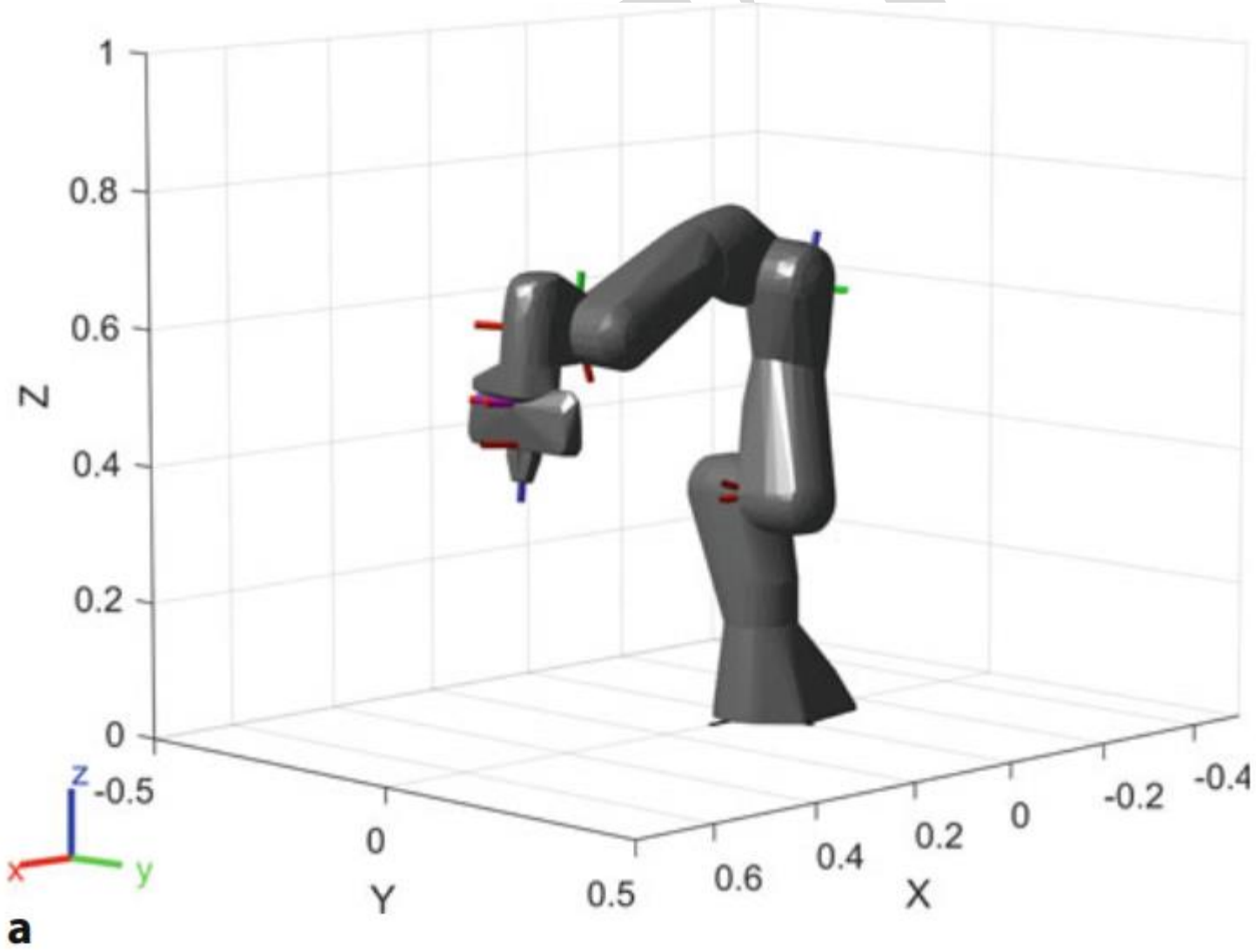
- For the 3D case we can create a robot model following a similar pattern. However, we can take some shortcuts.
- Firstly, we can create a rigidBodyTree robot directly from an ETS expression. The ets2rbt function has automatically partitioned the ETS expression and created link and joint objects.

```
>> a1 = 1; a2 = 1;
>> robot6 = ets2rbt(ETS3.Rz("q1")*ETS3.Ry("q2")* ...
>>    ETS3.Tz(a1)*ETS3.Ry("q3")*ETS3.Tz(a2)* ...
>>    ETS3.Rz("q4")*ETS3.Ry("q5")*ETS3.Rz("q6"));
```

DRAFT

- The second shortcut is to use one of the many models available in MATLAB.

```
>> loadrobot("<TAB>
abbIrb120
abbIrb120T
abbIrb1600
abbYuMi
amrPioneer3AT
amrPioneer3DX
amrPioneerLX
atlas
...
```



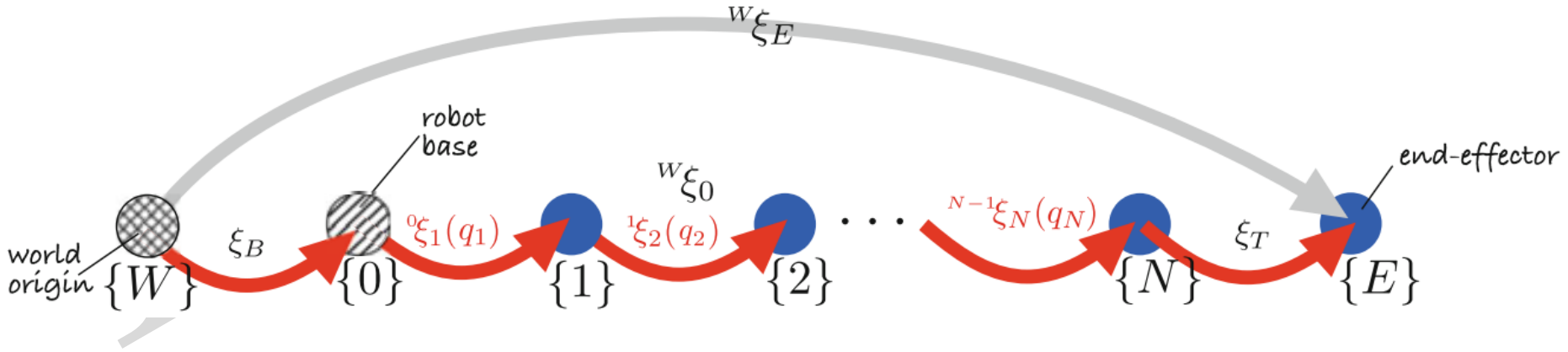
DRAFT

5.1.2.4 Tools and Bases

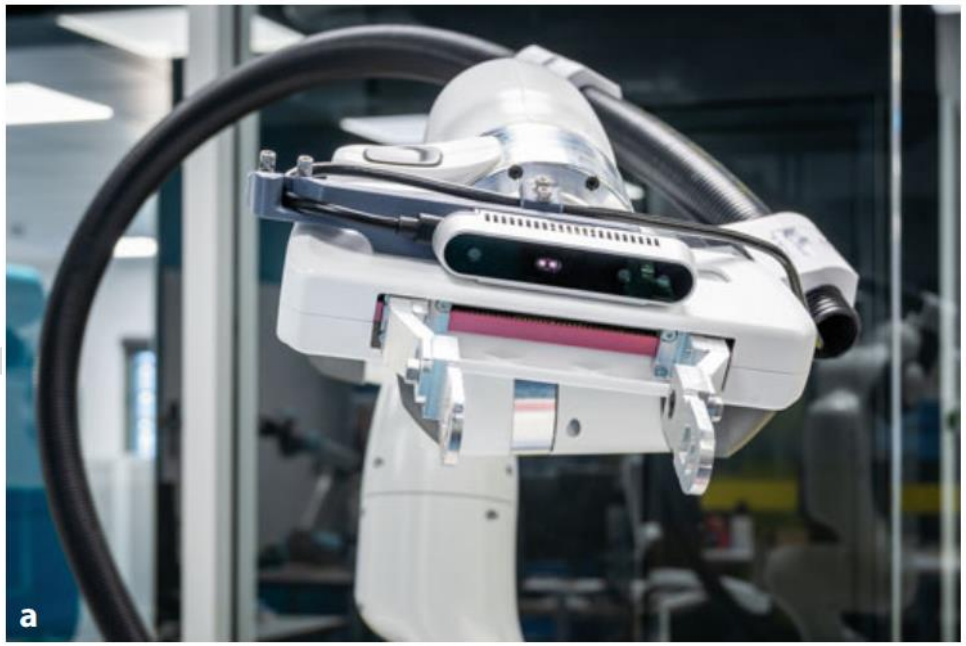
- It is useful to extend the forward kinematic expression by adding two extra transforms

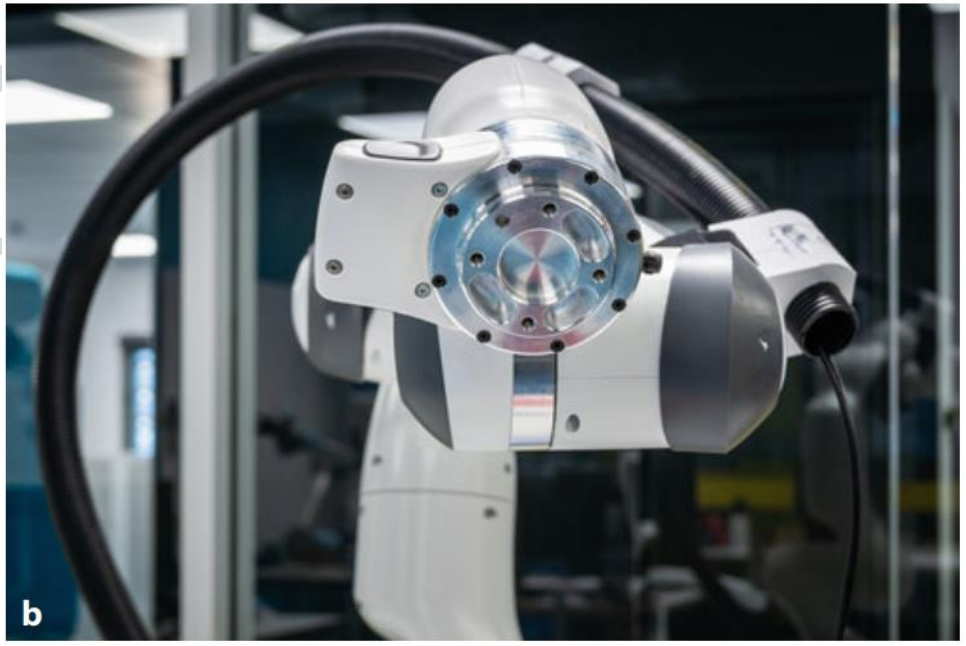
$$ {}^{W}\xi_{E} = \underbrace{{}^{W}\xi_{0}}_{\xi_B} \oplus {}^{0}\xi_{1} \oplus {}^{1}\xi_{2} \oplus \cdots \oplus {}^{N-1}\xi_{N} \oplus \underbrace{{}^{N}\xi_{E}}_{\xi_T} $$

- We have used W to denote the world frame.
- Conventionally, the start of the kinematic chain is the base of the robot, but the base transformation ξ_B allows us to place the base of the robot at an arbitrary pose within the world coordinate frame.



- There is no standard for kinematic models that dictates where the tool frame $\{N\}$ is physically located on the robot.
- The tool transformation ξ_T describes the pose of the tool tip – the bit that does the work and sometimes called the tool center point – with respect to frame $\{N\}$.





5.1.3 Branched Robots

- The simple robots discussed so far have a single end effector, but increasingly we see robots with multiple end effectors.
- Let's use the humanoid robot to illustrate key points of branched robots.
- The robot has a torso with four manipulator arms (two of which we call legs).



a

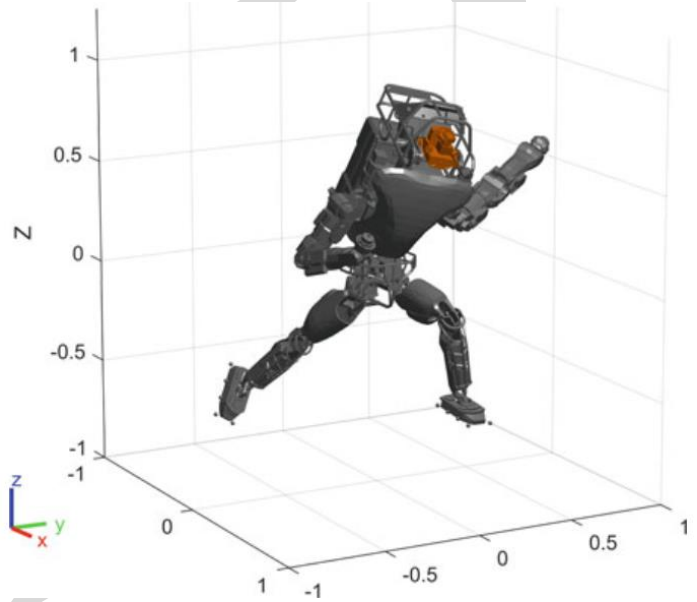

b

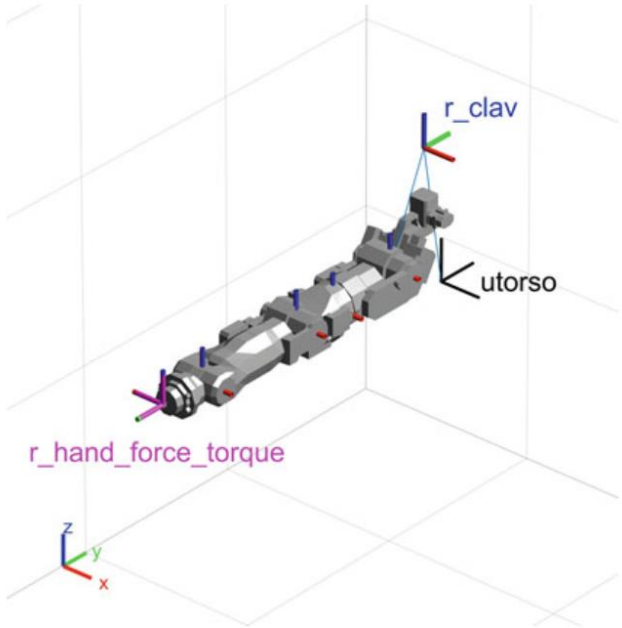
DRAFT

- We can load it into MATLAB and display it

```
>> atlas = loadrobot("atlas",DataFormat="row");
>> atlas.show;
```

- We see that multiple links including "l_clav", "r_clav", and "head" share "utorso" as their parent.
- We can extract the subtree of all links attached to a body.
- For forward kinematics, we now need to specify which end effector we are interested in.





DRAFT

5.1.4 Unifed Robot Description Format (URDF)

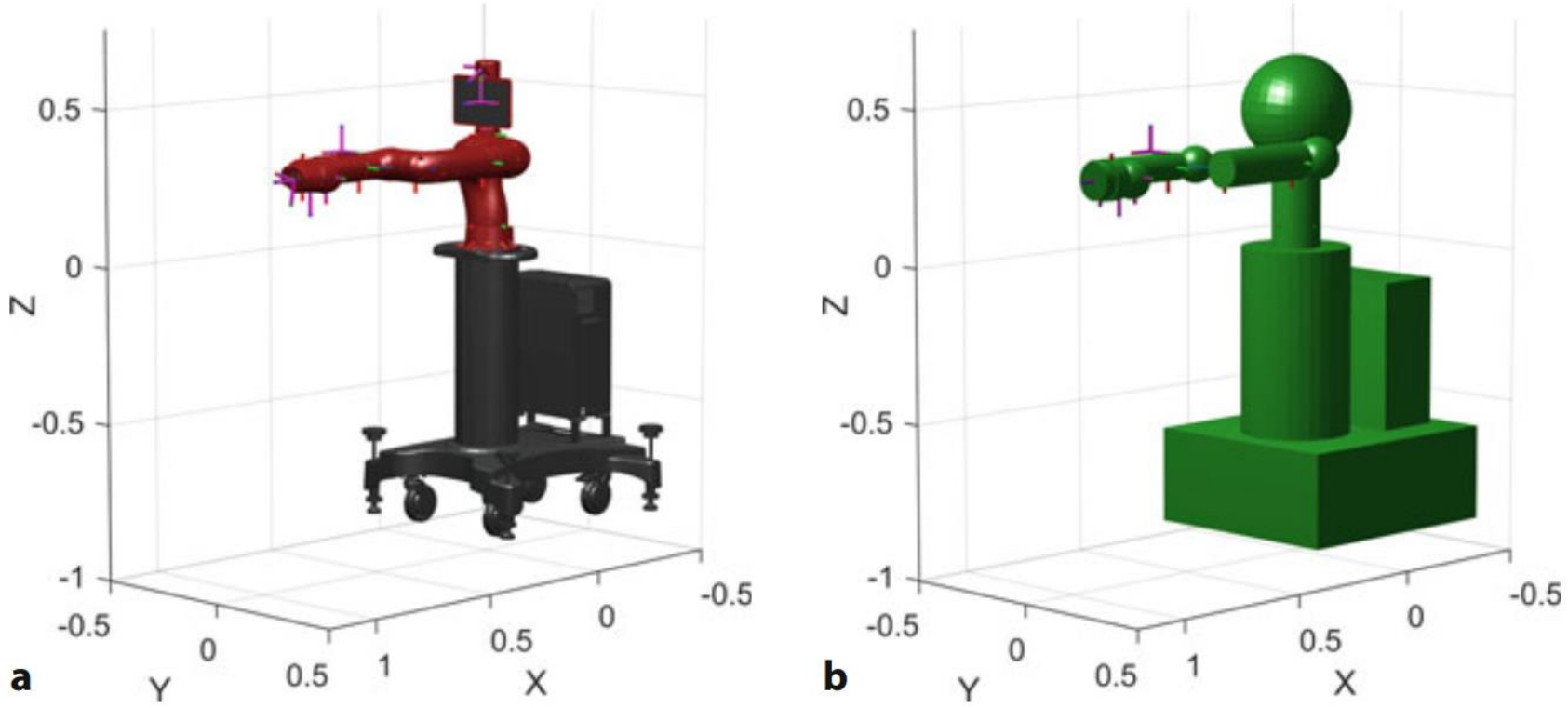
- So far we have used code to create rigid-body trees, but that is very specific to the programming language and software tools being used.
- To enable exchange of models we use URDF which is a portable XML-based file format.
- The format imported to extract a robot description, and models of many robots can be found online.
- We can parse a URDF file

```
>> sawyer = importrobot("sawyer.urdf");
```

which returns a rigid body tree initialized with parameters extracted from the URDF file.

DRAFT

- The geometry data allows for realistic 3D rendering of the robot.
- Each link is described by a mesh file which includes the 3D shape and optionally the surface color and texture.
- If available in the URDF file, collision meshes are also imported.
- Collision meshes are simple geometric approximations of the link shape that can be tested for intersection with other objects during simulation.



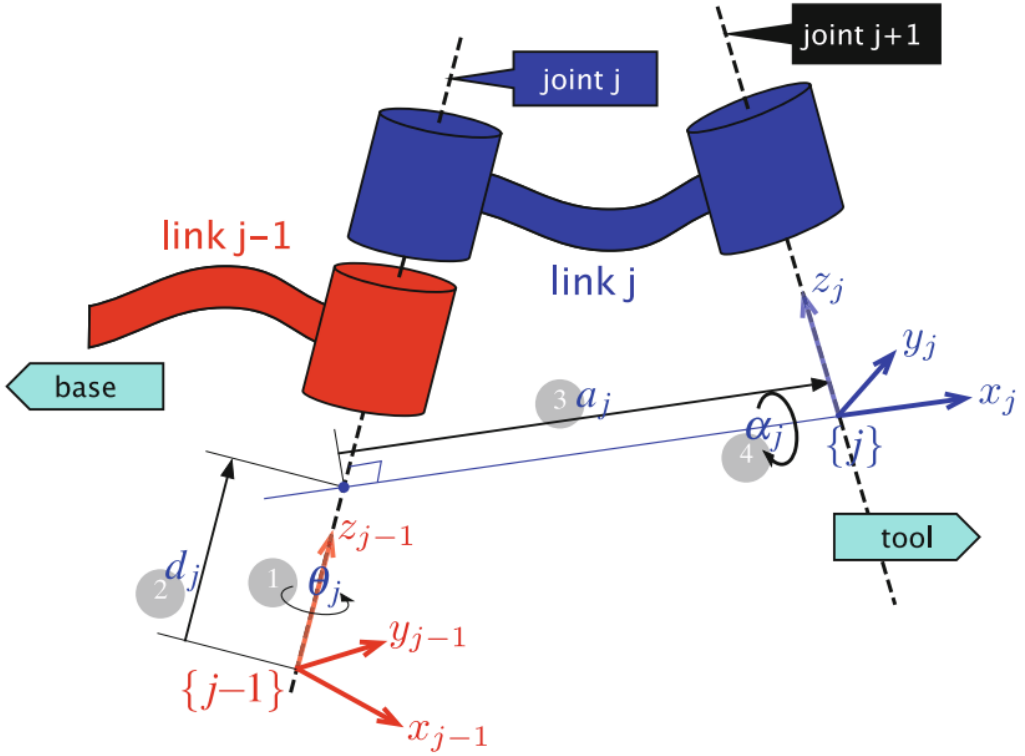
5.1.5 Denavit-Hartenberg Parameters

- From the 1960s, well before the advent of URDF, the way to share a robot model was as a simple table of numbers.
- There is one row per link-joint pair to describe the kinematics of the robot.
- This compact notation is known as Denavit-Hartenberg (DH) notation.

θ_j	d_j	a_j	α_j	σ_j
q_1	0.6718	0	90°	0
q_2	0	0.4318	0°	0
q_3	0.1505	0.0203	-90°	0
q_4	0.4318	0	90°	0
q_5	0	0	-90°	0
q_6	0	0	0°	0

DRAFT

- Each row in the table defines the spatial relationship between two consecutive link frames.
- A manipulator with N joints numbered from 1 to N has $N+1$ links, numbered from 0 to N.
- Joint j connects link $j-1$ to link j.
- It follows that link ℓ connects joint ℓ to joint $\ell+1$.
- Link 0 is the base of the robot, typically fixed and link N, the last link of the robot, carries the end effector or tool.



DRAFT

- Denavit-Hartenberg notation uses only four parameters but there are also two constraints: axis x_j intersects z_{j-1} and axis x_j is perpendicular to z_{j-1}.

Joint angle	θ_j	The angle between the $\boldsymbol{x}_{j-1}$ and $\boldsymbol{x}_j$ axes about the $\boldsymbol{z}_{j-1}$ axis	Revolute joint variable or constant
Link offset	d_j	The distance from the origin of frame {j − 1} to the $\boldsymbol{x}_j$ axis along the $\boldsymbol{z}_{j-1}$ axis	Prismatic joint variable or constant
Link length	a_j	The distance between the $\boldsymbol{z}_{j-1}$ and $\boldsymbol{z}_j$ axes along the $\boldsymbol{x}_j$ axis; for intersecting axes, a_j is parallel to $\boldsymbol{z}_{j-1} \times \boldsymbol{z}_j$	Constant
Link twist	α_j	The angle from the $\boldsymbol{z}_{j-1}$ axis to the $\boldsymbol{z}_j$ axis about the $\boldsymbol{x}_j$ axis	Constant
Joint type	σ_j	$\sigma = R$ for a revolute joint, $\sigma = P$ for a prismatic joint. By convention $R = 0$ and $P = 1$	Constant

- The transformation from link coordinate frame $\{j-1\}$ to frame $\{j\}$ is defined in terms of elementary transformations.

$$ {}^{j-1}\boldsymbol{\xi}_j = \boldsymbol{\xi}^{r_z}(\theta_j) \oplus \boldsymbol{\xi}^{t_z}(d_j) \oplus \boldsymbol{\xi}^{t_x}(a_j) \oplus \boldsymbol{\xi}^{r_x}(\alpha_j) $$

which can be expanded as an $\mathbf{SE}(3)$ matrix

$$ {}^{j-1}\mathbf{A}_j = \begin{pmatrix} \cos\theta_j & -\sin\theta_j \cos\alpha_j & \sin\theta_j \sin\alpha_j & a_j \cos\theta_j \\ \sin\theta_j & \cos\theta_j \cos\alpha_j & -\cos\theta_j \sin\alpha_j & a_j \sin\theta_j \\ 0 & \sin\alpha_j & \cos\alpha_j & d_j \\ 0 & 0 & 0 & 1 \end{pmatrix} $$

- In the Denavit-Hartenberg representation link 0 is the base of the robot and commonly for the first link $d_1 = 0$, but we could set $d_1 > 0$ to represent the height of the first joint above the world coordinate frame.
- For the final link, link N, the parameters d_N, a_N and α_N provide a limited means to describe the tool-tip pose with respect to the $\{N\}$ frame.

DRAFT

- Determining the Denavit-Hartenberg parameters for a particular robot is challenging, but Denavit-Hartenberg kinematic descriptions of many robots can be found in manufacturer datasheets and in the literature.
- They are very compact compared to a URDF model, and a table of Denavit-Hartenberg parameters is sufficient to compute the forward and inverse kinematics.

DRAFT

5.2 Inverse Kinematics

- A problem of real practical interest is the inverse of that just discussed: given the desired pose of the end effector ξ_E what are the required joint coordinates?
- For example, if we know the Cartesian pose of an object, what should the robot's joint coordinates be to reach it?
- This is the inverse kinematics problem which is written in functional form as

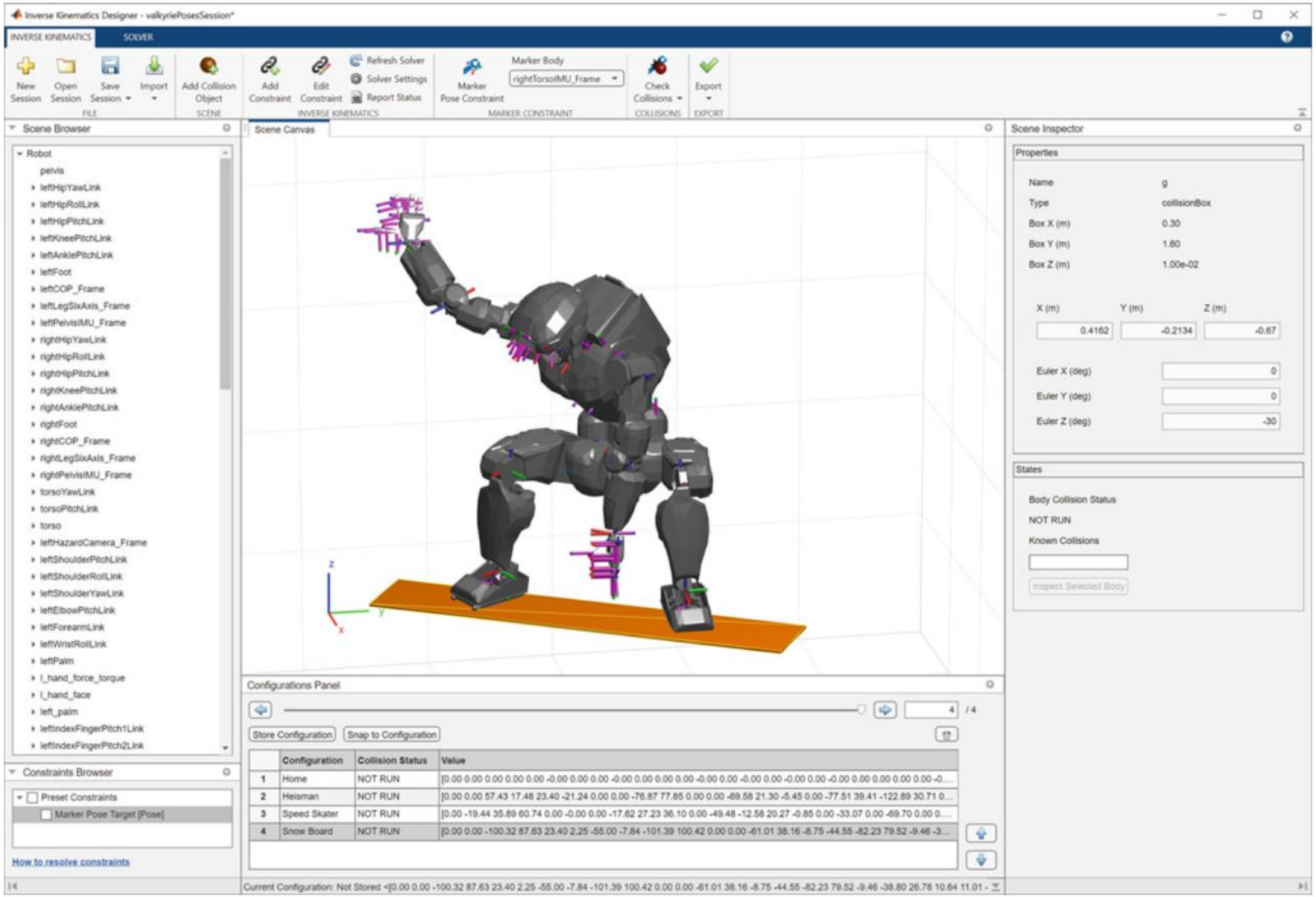
$$\boldsymbol{q} = \mathcal{K}^{-1}(\xi_E)$$

and, in general, is not unique, that is, a particular end-effector pose can be achieved by more than one joint configuration.

DRAFT

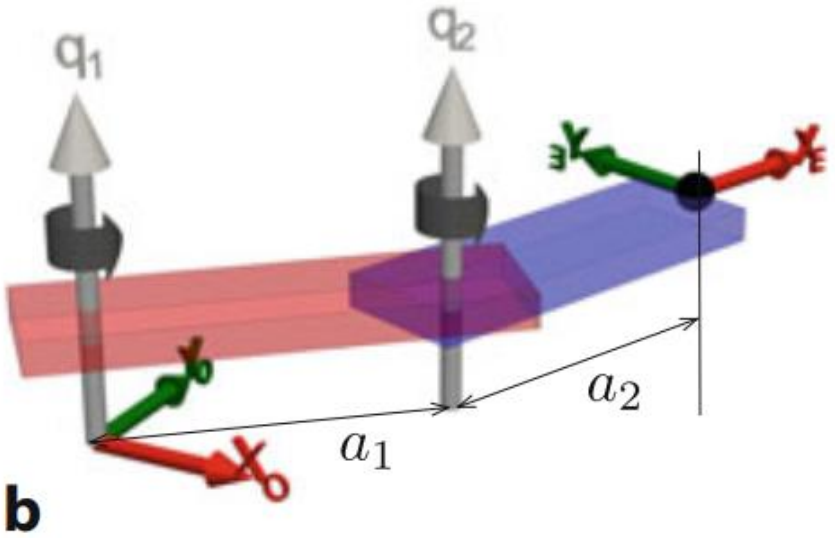
- Two approaches can be used to determine the inverse kinematics.
- Firstly, a closed-form or analytical solution can be found using geometric or algebraic techniques.
- However, this becomes increasingly challenging as the number of robot joints increases and for some serial-link manipulators no closed-form solution exists.
- Secondly, an iterative numerical solution can be used.
- We will see that determining an inverse kinematics solution depends on many constraints and parameters.
- Visual inspection of inverse kinematics results and the ability to store trajectories of joint configurations is critical.
- To assist with this task, we can use the interactive Inverse Kinematics Designer app (inverseKinematicsDesigner).

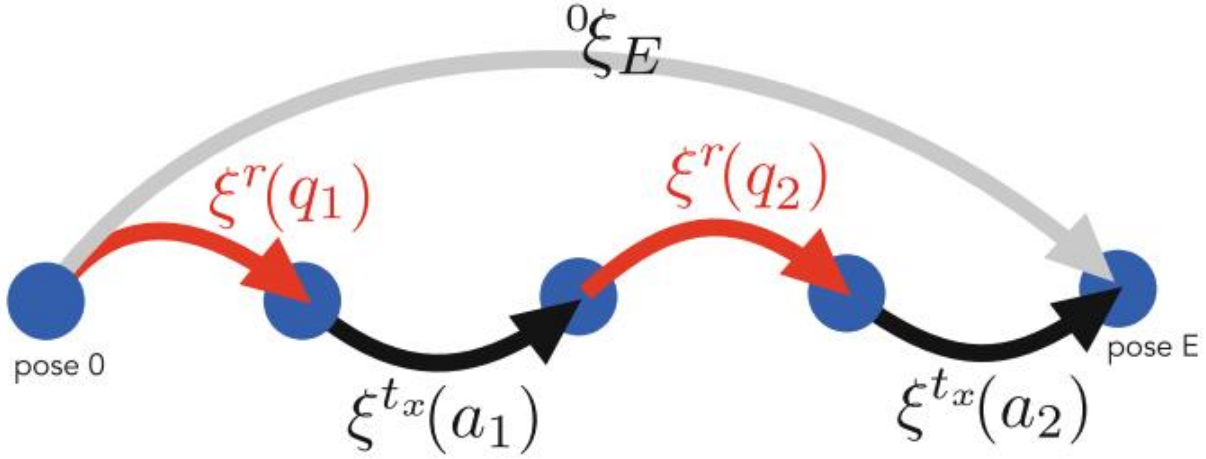
DRAFT


Inverse Kinematics Designer - valkyriePosesSession*
INVERSE KINEMATICS
SOLVER
New Session
Open Session
Save Session
Import
FILE
Add Collision Object
SCENE
Add Constraint
Edit Constraint
Refresh Solver
Solver Settings
Report Status
INVERSE KINEMATICS
Marker Pose Constraint
Marker Body
rightTorsoIMU_Frame
MARKER CONSTRAINT
Check Collisions
COLLISIONS
Export
EXPORT
Scene Browser
Robot
pelvis
leftHipYawLink
leftHipRollLink
leftHipPitchLink
leftKneePitchLink
leftAnklePitchLink
leftFoot
leftCOP_Frame
leftLegSixAxis_Frame
leftPelvisIMU_Frame
rightHipYawLink
rightHipRollLink
rightHipPitchLink
rightKneePitchLink
rightAnklePitchLink
rightFoot
rightCOP_Frame
rightLegSixAxis_Frame
rightPelvisIMU_Frame
torsoYawLink
torsoPitchLink
torso
leftHazardCamera_Frame
leftShoulderPitchLink
leftShoulderRollLink
leftShoulderYawLink
leftElbowPitchLink
leftForearmLink
leftWristRollLink
leftPalm
l_hand_force_torque
l_hand_face
left_palm
leftIndexFingerPitch1Link
leftIndexFingerPitch2Link
Constraints Browser
Preset Constraints
Marker Pose Target [Pose]
How to resolve constraints
Scene Canvas
Configurations Panel
4 / 4
Store Configuration
Snap to Configuration
Configuration
Collision Status
Value
1 Home NOT RUN
2 Heisman NOT RUN
3 Speed Skater NOT RUN
4 Snow Board NOT RUN
Current Configuration: Not Stored
Scene Inspector
Properties
Name g
Type collisionBox
Box X (m) 0.30
Box Y (m) 1.60
Box Z (m) 1.00e-02
X (m) Y (m) Z (m)
0.4162 -0.2134 -0.67
Euler X (deg) 0
Euler Y (deg) 0
Euler Z (deg) -30
States
Body Collision Status
NOT RUN
Known Collisions
Inspect Selected Body

5.2.1 2-Dimensional (Planar) Robotic Arms

- We will solve the inverse kinematics for the 2-joint robot in two ways: algebraic closed-form and numerical.





5.2.1.1 Closed-Form Solution

- We start by computing the forward kinematics symbolically.
- We define some symbolic constants for the robot's lengths and then create an ETS for the robot

```
>> syms a1 a2 real
>> e = ETS2.Rz("q1")*ETS2.Tx(a1)*ETS2.Rz("q2")*ETS2.Tx(a2);
```

- Next, we define some symbolic variables to represent the joint angles

```
>> syms q1 q2 real
```

and then compute the forward kinematics

```
>> TE = e.fkine([q1 q2])
TE =
[cos(q1 + q2), -sin(q1 + q2), a2*cos(q1 + q2) + a1*cos(q1)]
[sin(q1 + q2),  cos(q1 + q2), a2*sin(q1 + q2) + a1*sin(q1)]
[           0,             0,                           1]
>> transl = TE(1:2,3)';
```

DRAFT

- Finally, we define two more symbolic variables to represent the desired endeffector position (x, y)

```
>> syms x y real
```

and equate them with the results of the forward kinematics

```
>> e1 = x == transl(1)
e1 =
x == a2*cos(q1 + q2) + a1*cos(q1)
>> e2 = y == transl(2)
e2 =
y == a2*sin(q1 + q2) + a1*sin(q1)
```

which gives two scalar equations that we can solve simultaneously

```
>> [s1,s2] = solve([e1 e2],[q1 q2]);
```

where the arguments are respectively the set of equations and the set of unknowns to solve for. The outputs are the solutions for q1 and q2 respectively.

DRAFT

- We observed that two (or more) different sets of joint angles give the same end-effector position, and this means that the inverse kinematics does not have a unique solution.

```
>> length(s2)
ans =
     2
```

indicating two solutions.

- One solution for q_2 for a given link length $a_1 = 1$ and $a_2 = 1$ is

```
>> subs(s2(2),[a1 a2],[1 1])
ans =
2*atan((-(x^2 + y^2)*(x^2 + y^2 - 4))^(1/2)/(x^2 + y^2))
```

and would be used in conjunction with the corresponding element of the solution vector for q_1, which is s1(2).

DRAFT

- $q_1 = 30^o$ and $q_2 = 40^o$ result in an end-effector position of $x = 1.21$ and $y = 1.44$.
- Let's see if one of our algebraic solutions matches the original q1 and q2 angles.

```
>> xfk = eval(subs(transl(1), [a1 a2 q1 q2], ...
>>    [1 1 deg2rad(30) deg2rad(40)]))
xfk =
    1.2080
>> yfk = eval(subs(transl(2), [a1 a2 q1 q2], ...
>>    [1 1 deg2rad(30) deg2rad(40)]))
yfk =
    1.4397
```

- Substituting these values and the known link lengths a_1 and a_2 allows us to numerically evaluate the required q_1 and q_2 angles to reach that configuration.

```
>> q1r = eval(subs(s1(2),[a1 a2 x y],[1 1 xfk yfk]));
>> q1 = rad2deg(q1r)
q1 =
   30.0000
>> q2r = eval(subs(s2(2),[a1 a2 x y],[1 1 xfk yfk]));
>> q2 = rad2deg(q2r)
q2 =
   40.0000
```

DRAFT

5.2.1.2 Numerical Solution

- We can think of the inverse kinematics problem as that of adjusting the joint coordinates until the forward kinematics matches the desired pose.
- More formally this is an optimization problem – to minimize the error between the forward kinematic solution and the desired end-effector pose ξ_E^*.

$$\boldsymbol{q}^* = \arg\min_{\boldsymbol{q}} \| \mathcal{K}(\boldsymbol{q}) \ominus \boldsymbol{\xi}_E^* \|$$

- We create an ETS for the robot

```
>> e = ETS2.Rz("q1")*ETS2.Tx(1)*ETS2.Rz("q2")*ETS2.Tx(1);
```

and define an error function based on the end-effector position error, not its orientation

$$E(q) = \| [\mathcal{K}(\boldsymbol{q})]_t - (x^*, \ y^*)^\top \|$$

DRAFT

- We can solve this using the multi-variable minimization function fminsearch

```
>> pstar = [0.6 0.7];  % Desired position
>> q = fminsearch(@(q) norm(se2(e.fkine(q)).trvec-pstar),[0 0])
q =
   -0.2295    2.1833
```

where the first argument is the error function; and the second argument is the initial guess at the joint coordinates.

- The computed joint angles indeed give the desired end-effector position

```
>> printtform2d(e.fkine(q),unit="deg")
t = (0.6, 0.7), 112 deg
```

- there are two solutions for q, but the solution that is found using this approach depends on the initial guess of q passed to the minimizer.

DRAFT

5.2.2 3-Dimensional Robotic Arms

DRAFT

5.2.2.1 Closed-Form Solution

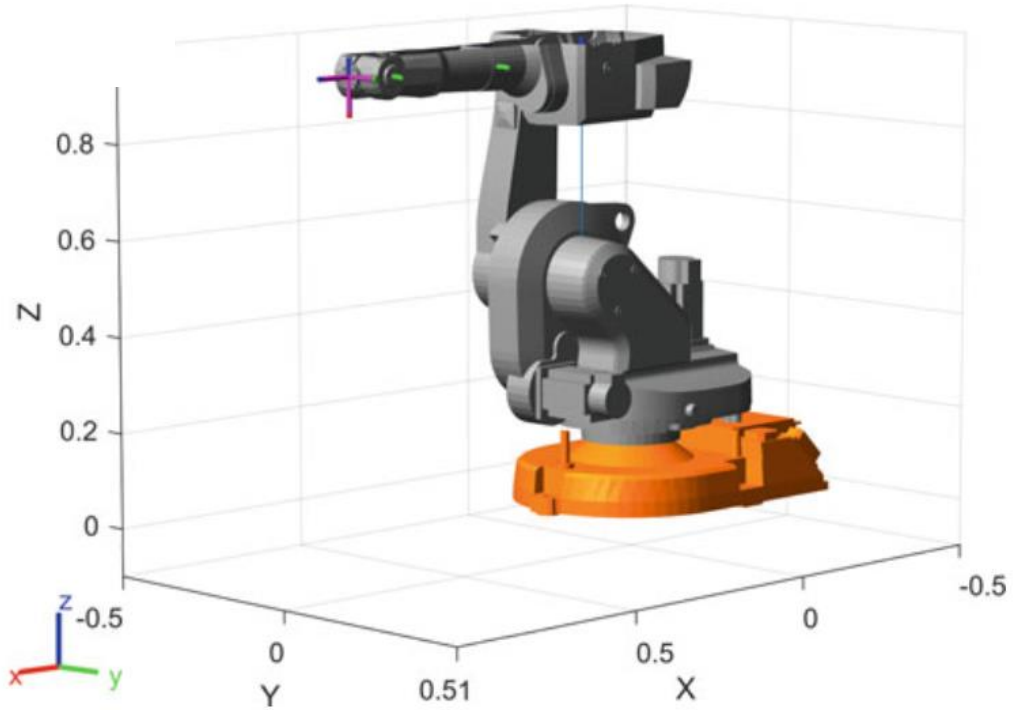
- Closed-form solutions have been developed for most common types of 6-axis industrial robots, typically those defined by Denavit-Hartenberg parameters.
- A necessary condition for a closed-form inverse kinematics (also called analytical inverse kinematics) solution of a 6-axis robot is a spherical wrist mechanism.
- Some robot a humanoid robot. models have closed-form inverse kinematics and we will illustrate this for the ABB IRB 1600 industrial robot.

DRAFT

- Closed-form solutions have been developed for most common types of 6-axis industrial robots, typically those defined by Denavit-Hartenberg parameters.

```
>> abb = loadrobot("abbIrb1600",DataFormat="row");
>> abb.show;
>> abb.showdetails

Robot: (7 bodies)
 Idx   Body Name      Joint Name    Joint Type
 ---   ---------      ----------    ----------
   1      link_1         joint_1      revolute
   2      link_2         joint_2      revolute
   3      link_3         joint_3      revolute
   4      link_4         joint_4      revolute
   5      link_5         joint_5      revolute
   6      link_6         joint_6      revolute
   7       tool0   joint_6-tool0         fixed
---------------------
```



- Before we can find a closed-form solution, we need to generate an inverse kinematics function specific to this robot

```
>> abbIKFcn = aIK.generateIKFunction("ikIRB1600")
abbIKFcn =
  function_handle with value:
    @ikIRB1600
```

- For a given pose of body tool0, we can now find the closed-form solutions for the joint angles

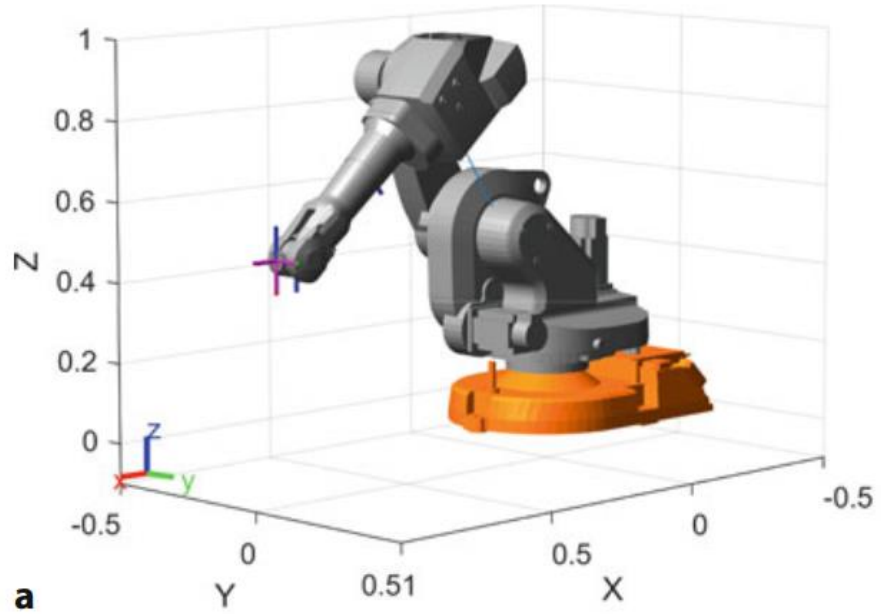
```
>> tgtPose = trvec2tform([0.93 0 0.5])*eul2tform([0 pi/2 0]);
>> qsol = abbIKFcn(tgtPose)
qsol =
    0.0000    0.5695    0.1306    3.1416    0.7000    3.1416
    0.0000    0.5695    0.1306         0   -0.7000    0.0000
```

- These two configurations are both physically achievable within the joint limits defined by the abb robot.

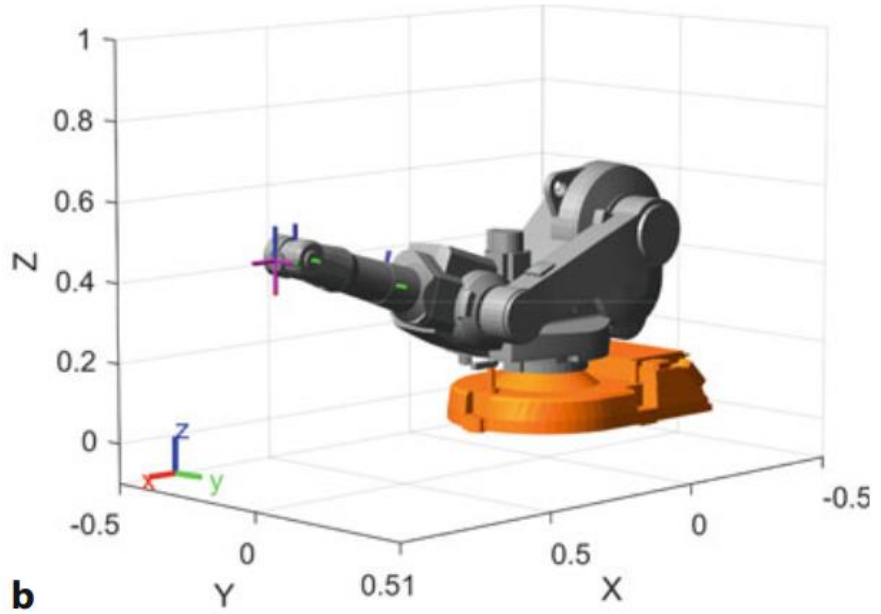
DRAFT

- In general, there are many more configurations that give the same end-effector pose.
- We can retrieve all of them by passing another false argument to the function, which indicates that joint limits should be ignored

```
>> qsol = abbIKFcn(tgtPose,false)
qsol =
    0.0000    2.5344    3.0110    0.0000    0.7378    0.0000
   -3.1416    4.3471   -0.8952    3.1416    0.3103    0.0000
    0.0000    0.5695    0.1306    3.1416    0.7000    3.1416
    ...
```



a



b

- At a singularity, the alignment of axes reduces the effective degrees of freedom (the gimbal lock problem again).
- The IRB 1600 has a wrist singularity when q_5 is equal to zero and the axes of joints 4 and 6 become aligned.
- In this case the best that the closed-form solution can do is to constrain $q_4 + q_6$ but their individual values are arbitrary.
- For example, consider the configuration

```
>> q = [0 pi/4 0 0.1 0 0.4];
>> qsol = abbIKFcn(abb.getTransform(q,"tool0"))
qsol =
    0.0000    0.7854    0.0000   -0.5354         0    1.0354
```

which has quite different values for q_4 and q_6 but the sum $q_4 + q_6 = 0.5$ is the same for both cases.

DRAFT

5.2.2.2 Numerical Solution

- For the case of robots which do not have six joints and a spherical wrist or for which no analytical solution is available, we need to use an iterative numerical solution.
- we use the inverseKinematics object to compute the inverse kinematics solution numerically

```
>> abbHome = abb.homeConfiguration;
>> T = abb.getTransform(qn,"tool0");
>> printtform(T)
T: t = (0.93, 0, 0.5), RPY/zyx = (0, 1.57, 0) rad
>> abbIK = inverseKinematics(RigidBodyTree=abb);
```

DRAFT

```
>> [qsol,solinfo] = abbIK("tool0",T,ones(1,6),abbHome)
qsol =
   -0.0000    0.5695    0.1306    0.0000   -0.7000    0.0000

solinfo =
  struct with fields:
          Iterations: 13
    NumRandomRestarts: 0
        PoseErrorNorm: 6.6805e-09
             ExitFlag: 1
               Status: 'success'
```

which indicates success after just 13 iterations and with a very low solution error or residual (see the field PoseErrorNorm).

- The third input to the abbIK function is the vector of weights for orientation and position. This tells the optimizer what aspect of the final pose to prioritize when calculating the cost.

- The joint configuration however is different from the original value

```
>> qn
qn =
    0.0000    0.5695    0.1306    3.1416    0.7000    3.1416
```

but does result in the correct end-effector pose

```
>> T2 = abb.getTransform(qsol,"tool0");
>> printtform(T2)
T2: t = (0.93, 0, 0.5), RPY/zyx = (0, 1.57, 0) rad
```

- A limitation of this general numeric approach is that it does not provide explicit control over which solution is found as the analytical approach did – the only control is implicit via the initial estimate of joint coordinates (we used the home configuration with all zeros).

DRAFT

- If we specify different initial joint coordinates

```
>> qsol = abbIK("tool0",T,ones(1,6),[0 0 0 pi 0 0])
qsol =
    0.0000    0.5695    0.1306    3.1416    0.7000   -3.1416
```

we have nudged the minimizer to converge on the other valid configuration.

- As expected, the general numerical approach of inverseKinematics is considerably slower than the analytical approach of analyticalInverseKinematics.
- The analytical approach is essentially a constant time operation for a given robot while the numerical approach is iterative and its runtime depends strongly on the initial guess.
- However, the numerical solver has the great advantage of being able to work with manipulators at singularities and for manipulators with less than, or more than, six joints.

DRAFT

5.3 Trajectories

- A very common requirement in robotics is to move the end effector smoothly from one pose to another.
- We will discuss two approaches to generating such trajectories: in configuration space and in task space. These are known respectively as joint-space and Cartesian motion.

DRAFT

5.3.1 Joint-Space Motion

- Consider the end effector moving between two poses

```
>> TE1 = se3(3,"rotx",[0.6 -0.5 0.1]);
>> TE2 = se3(2,"rotx",[0.4 0.5 0.1]);
```

which describe two points in the xy-plane with different end-effector orientations.

- The joint configurations for these poses are

```
>> sol1 = abbIKFcn(TE1.tform);
>> sol2 = abbIKFcn(TE2.tform);
>> waypts = [sol1(1,:)' sol2(1,:)'];
```

- We require the motion to occur over a time period of 2 seconds in 20 ms time steps, and we create an array of time values

```
>> t = 0:0.02:2;
```

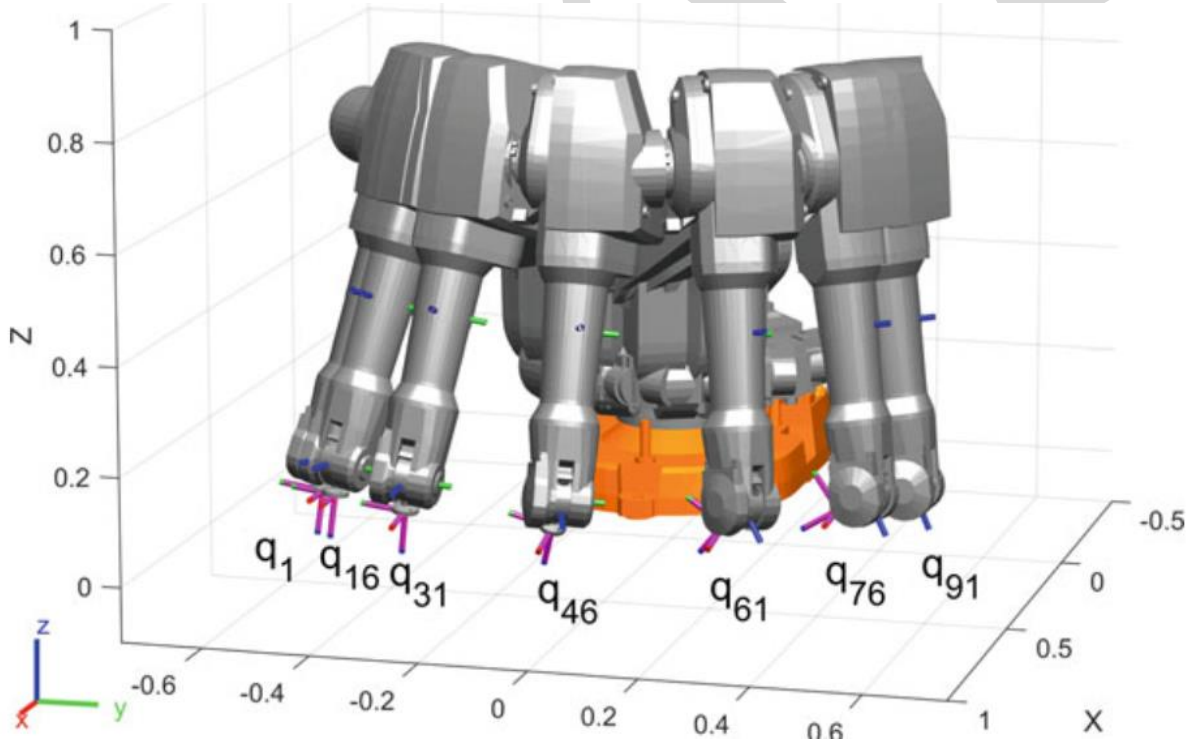
DRAFT

- A joint-space trajectory is formed by smoothly interpolating between the two joint configurations. We can either interpolate the joint angles with a 5th-order polynomial

```
>> [q,qd,qdd] = quinticpolytraj(waypts,[0 2],t);
```

or apply a trapezoidal velocity profile

```
>> [q,qd,qdd] = trapveltraj(waypts,numel(t),EndTime=2);
```

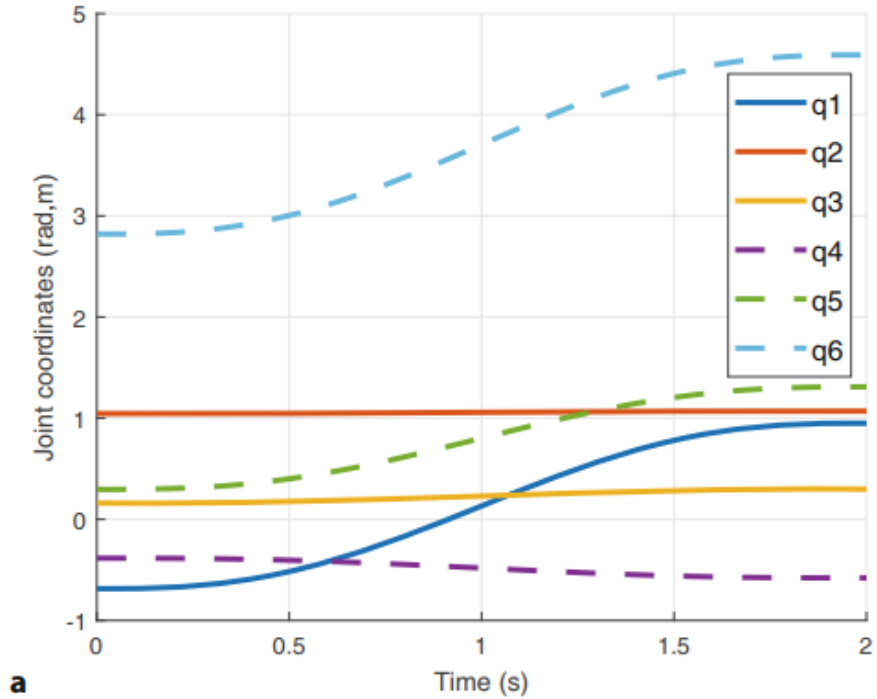


DRAFT

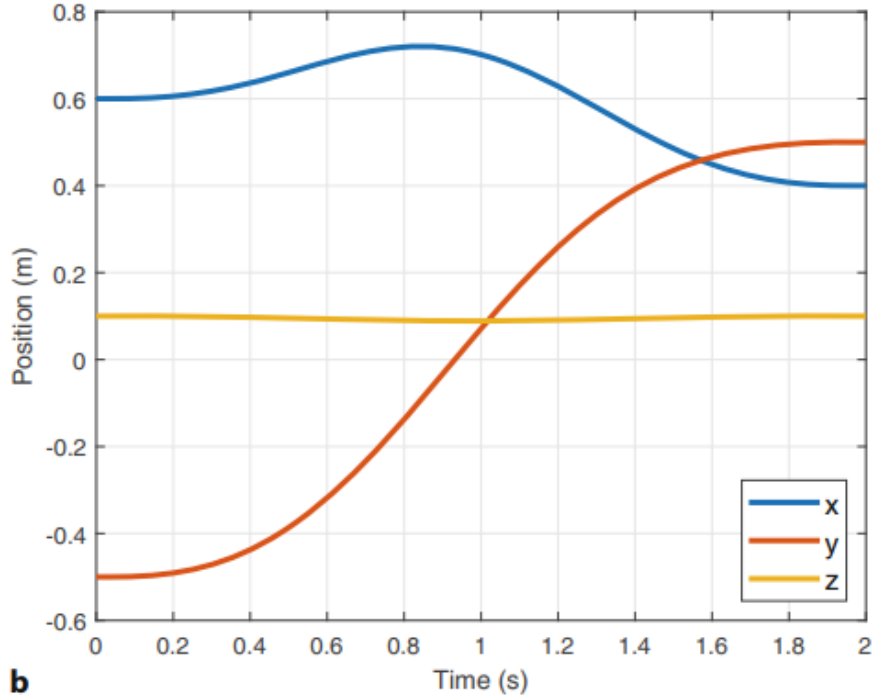
- The joint-space trajectory is smooth, which was one of our criteria, but we do not know how the robot's end effector will move in Cartesian space.
- We can easily determine this by applying forward kinematics to the joint coordinate trajectory

```
>> for i = 1:size(q,2)
>>    trajT(i) = se3(abb.getTransform(q(:,i)',"tool0"));
>> end
>> size(trajT)
ans =
     1   101
```

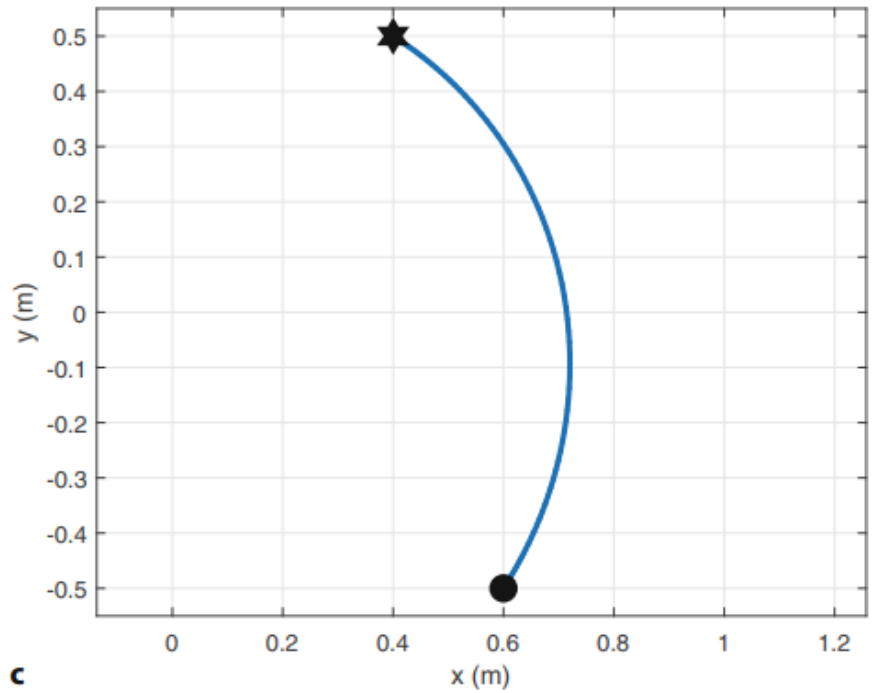
DRAFT


Joint coordinates (rad,m)
Time (s)
q1
q2
q3
q4
q5
q6

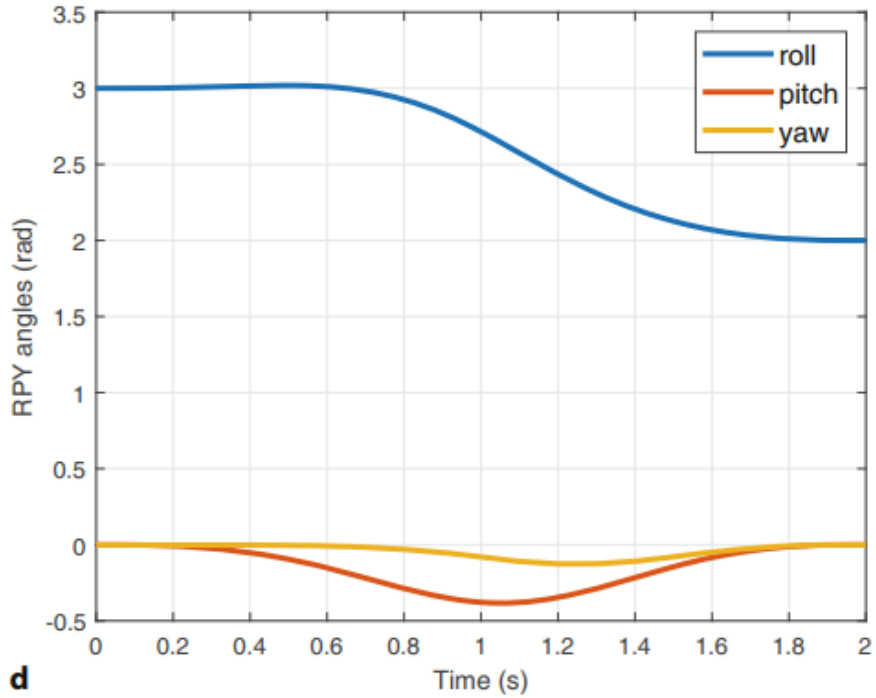
a


Position (m)
Time (s)
x
y
z

b


y (m)
x (m)

c


RPY angles (rad)
Time (s)
roll
pitch
yaw

d

5.3.2 Cartesian Motion

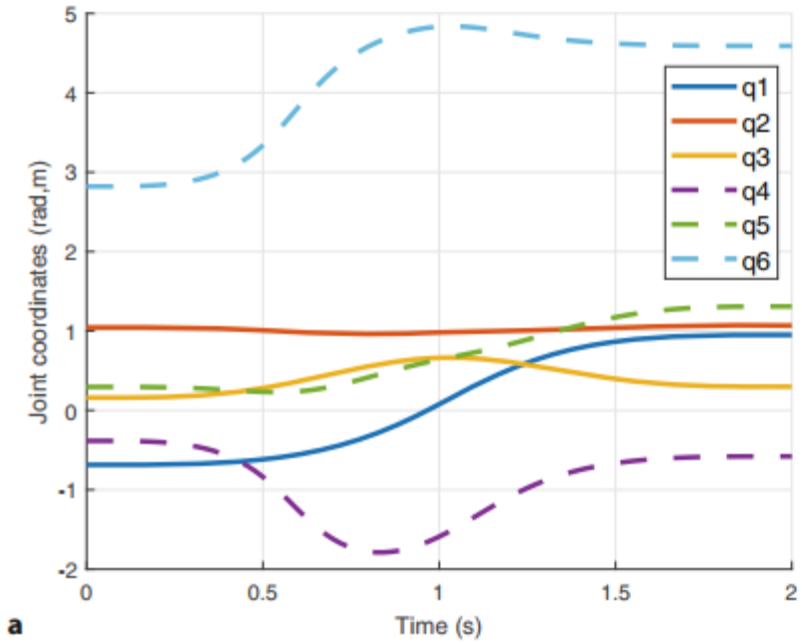
- For many applications we require straight-line motion in Cartesian space which is known as Cartesian motion. This is implemented by

```
>> T = transformtraj(TE1,TE2,[0 2],t);
```

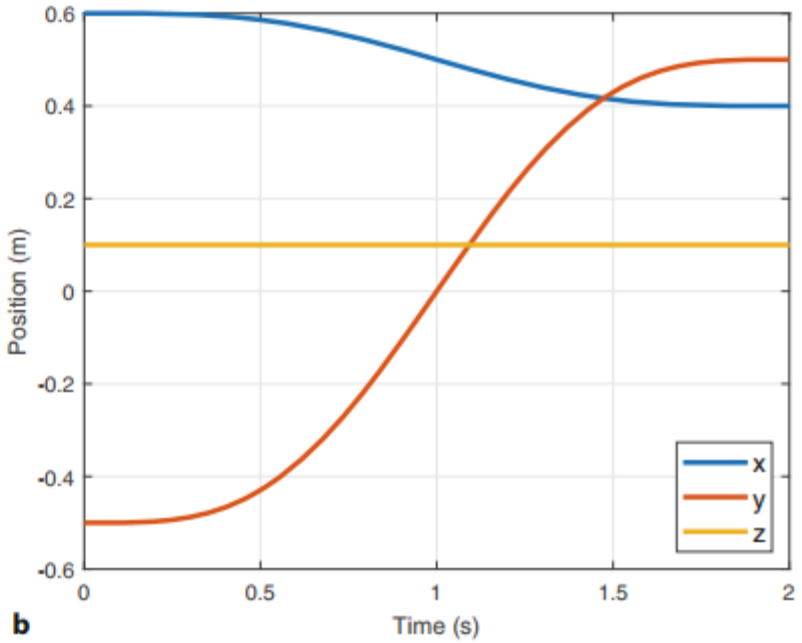
where the arguments are the initial and final pose and the time vector.

- The corresponding joint-space trajectory is obtained by applying the analytical inverse kinematics repeatedly.
- It is noteworthy that for trajectories, we want to ensure that there are no sudden jumps in joint angles.
- To accomplish that we can pass additional arguments to abbIKFcn to sort multiple solutions based on the distance to the previous trajectory configuration.

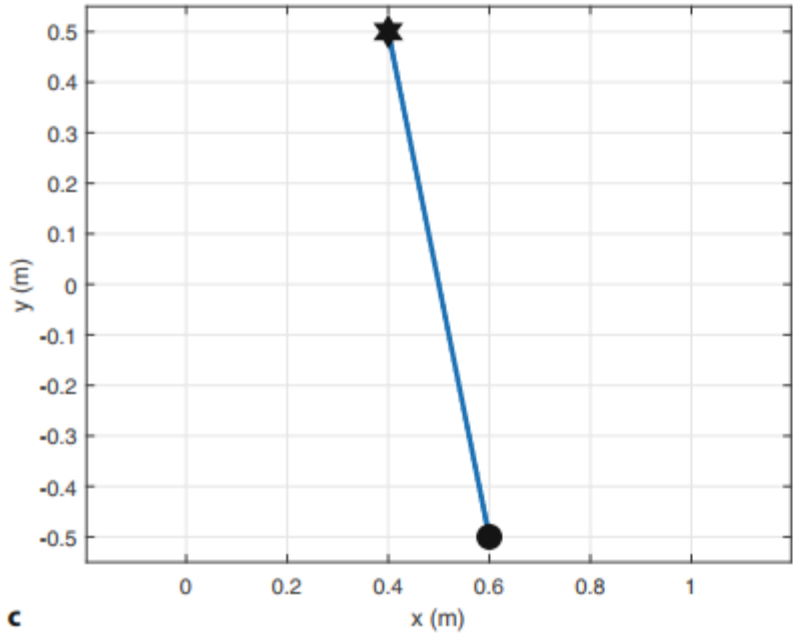
DRAFT


Joint coordinates (rad,m)
Time (s)
q1
q2
q3
q4
q5
q6

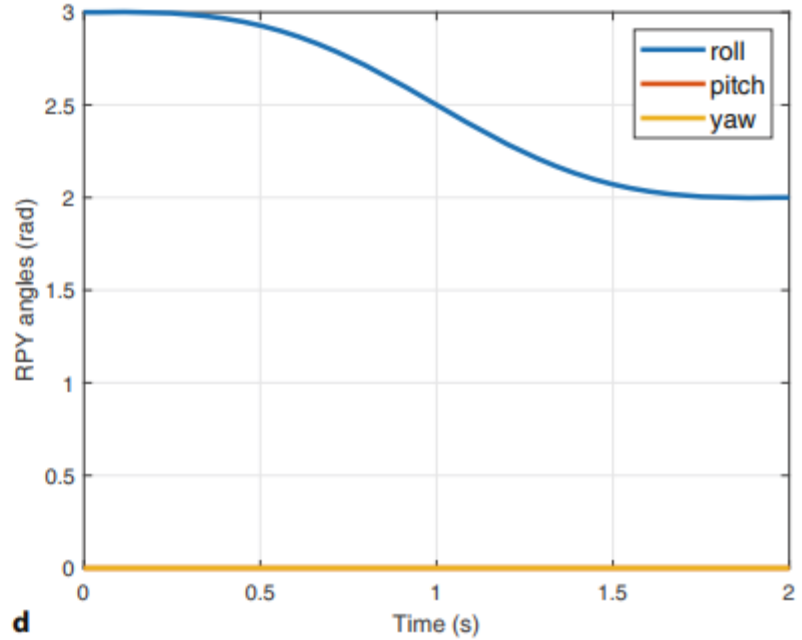
a


Position (m)
Time (s)
x
y
z

b


y (m)
x (m)

c


RPY angles (rad)
Time (s)
roll
pitch
yaw

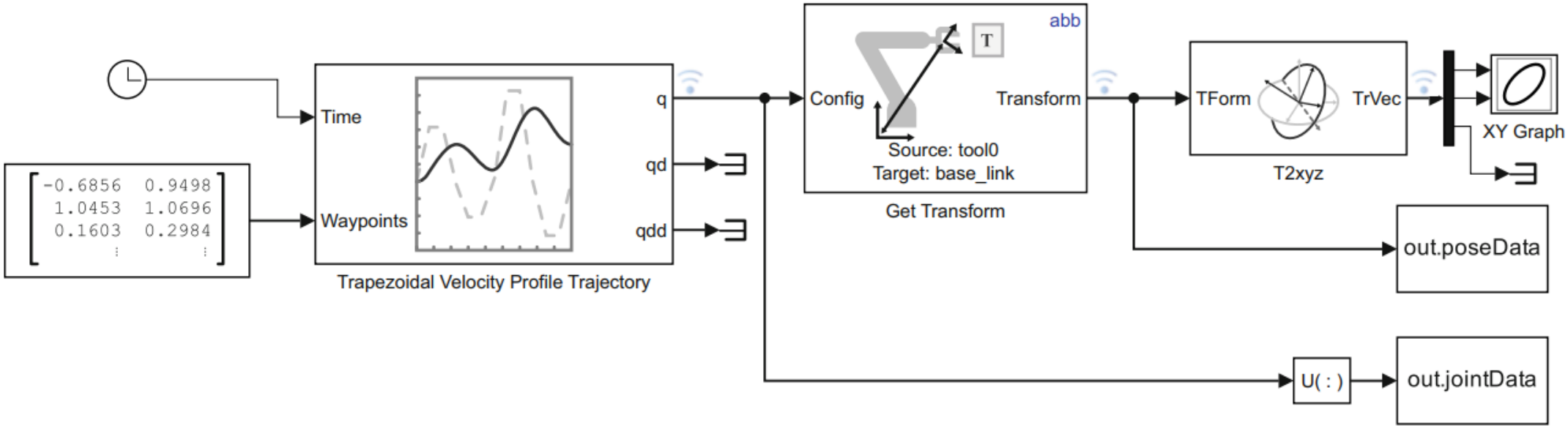
d

5.3.3 Kinematics in Simulink

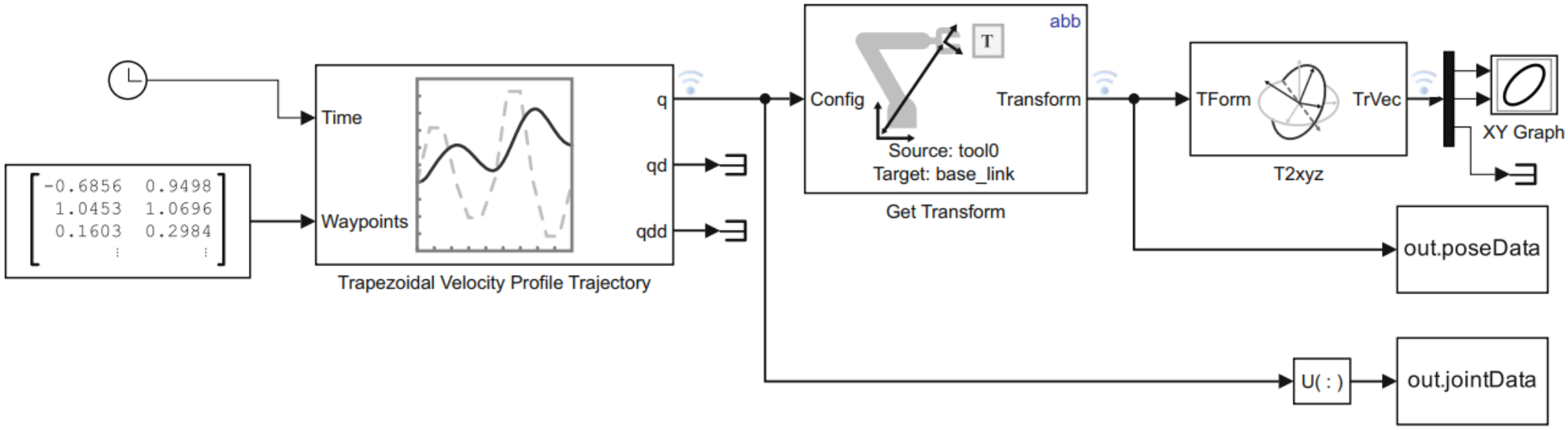
- We can also implement the example using Simulink®

```
>> sl_jointspace
```

- The inputs of the Trapezoidal Velocity Profile Trajectory block are the time and the initial and final values for the joint coordinates as columns.



- The smoothly varying joint angles are wired to a Get Transform block to compute the forward kinematics.
- This block has a parameter to specify the robot object to be used, in this case, set to abb.
- The Cartesian position of the end-effector pose is extracted using the T2xyz block which is analogous to the tform2trvec function.
- The XY Graph block plots y against x.



5.3.4 Motion Through a Singularity

- In this next example we deliberately choose a trajectory that moves through a robot wrist singularity.
- We repeat the previous example but choose different start and end poses relative to a known singular configuration qsing

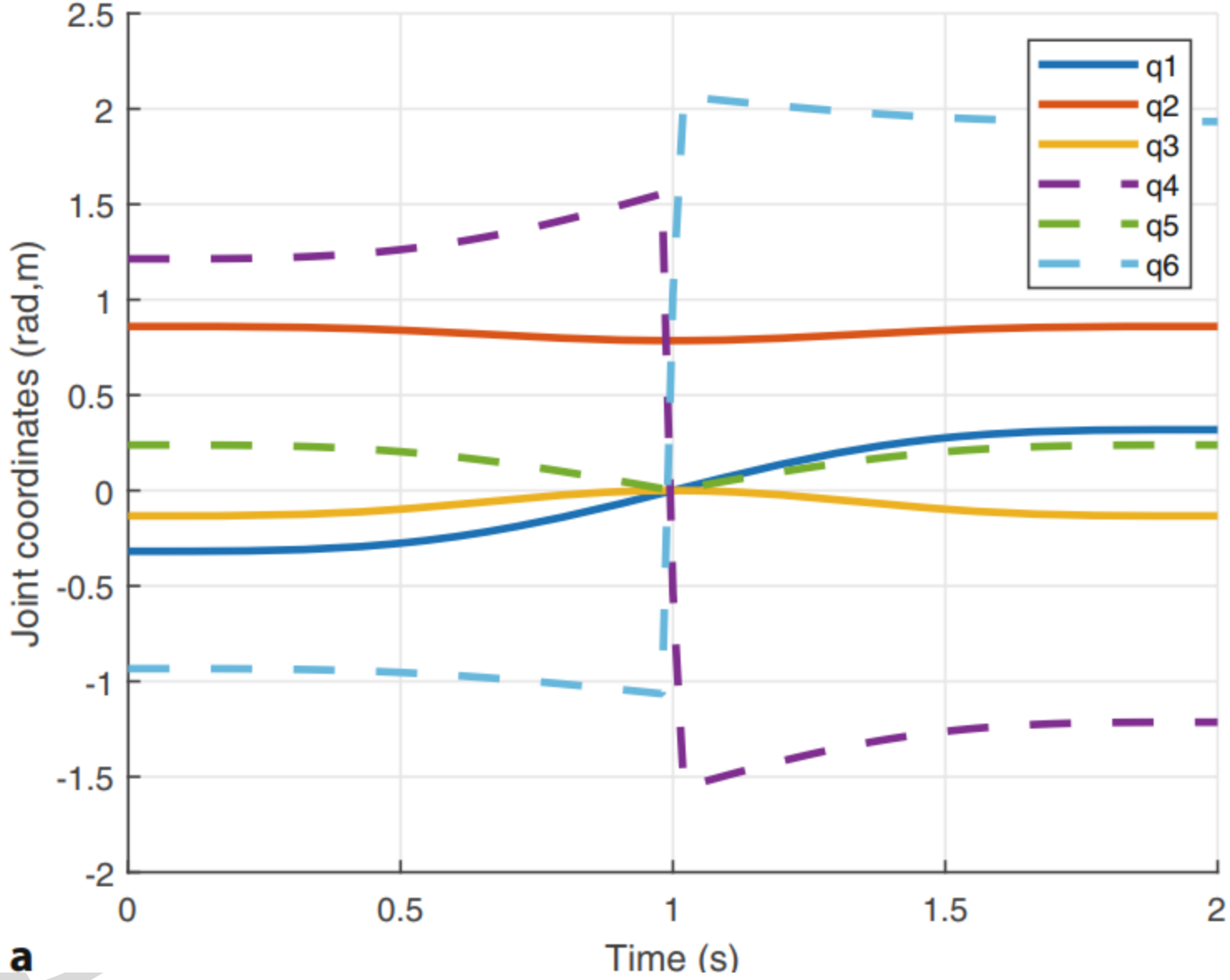
```
>> qsing = [0 pi/4 0 0.1 0 0.4];
>> TG = se3(abb.getTransform(qsing,"tool0"));
>> TE1 = se3(trvec2tform([0 -0.3 0]))*TG;
>> TE2 = se3(trvec2tform([0 0.3 0]))*TG;
```

which results in motion in the y-direction.

- The Cartesian path and joint coordinates are

```
>> Ts = transformtraj(TE1,TE2,[0 2],t, ...
>>   TimeScaling=[s;sd;sdd]);
>> qj = ikineTraj(abbIKFcn,Ts);
```

DRAFT


2.5
2
1.5
1
0.5
0
-0.5
-1
-1.5
-2
Joint coordinates (rad,m)
0
0.5
1
1.5
2
Time (s)
q1
q2
q3
q4
q5
q6

a

- At time $t \approx 1s$ we observe that the wrist joint angles q_4 and q_6 change very rapidly.
- At this time q_5 is close to zero which means that the q_4 and q_6 rotational axes of the wrist are almost aligned – another gimbal lock situation or singularity.
- The robot has lost one degree of freedom and is now effectively a 5- axis robot.
- This axis alignment means that we can only solve for the sum q_4 and q_6 meaning that there are an infinite number of solutions for q_4 and q_6 that have this sum.

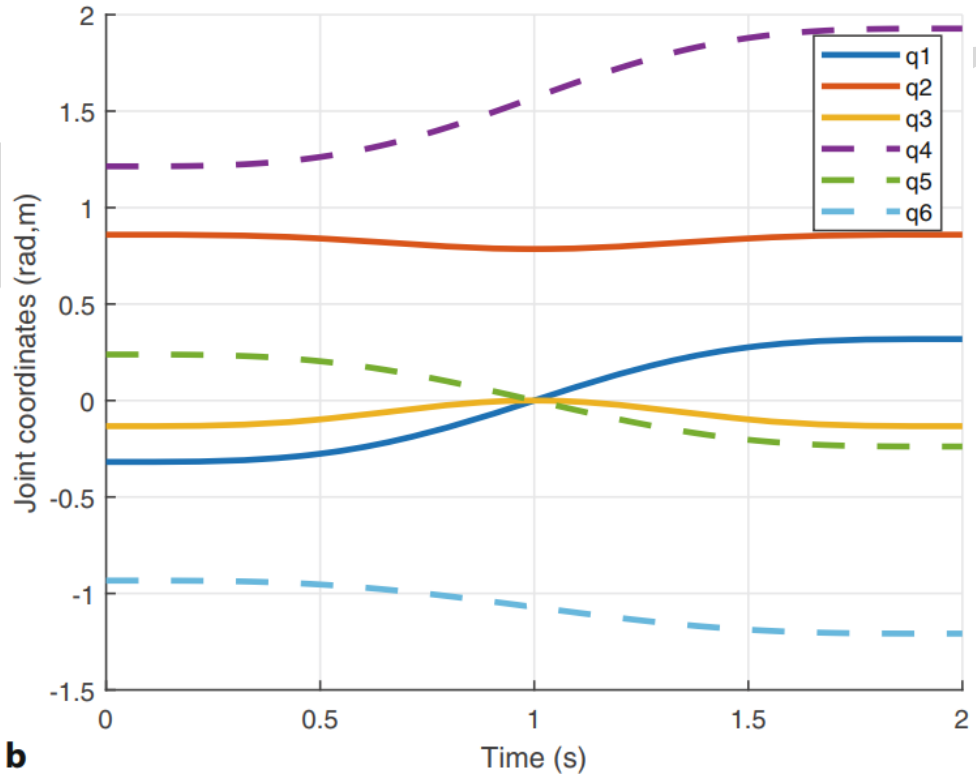
DRAFT

- We observe that numerical inverse kinematics

```
>> qj_num = ikineTrajNum(abb,Ts);
```

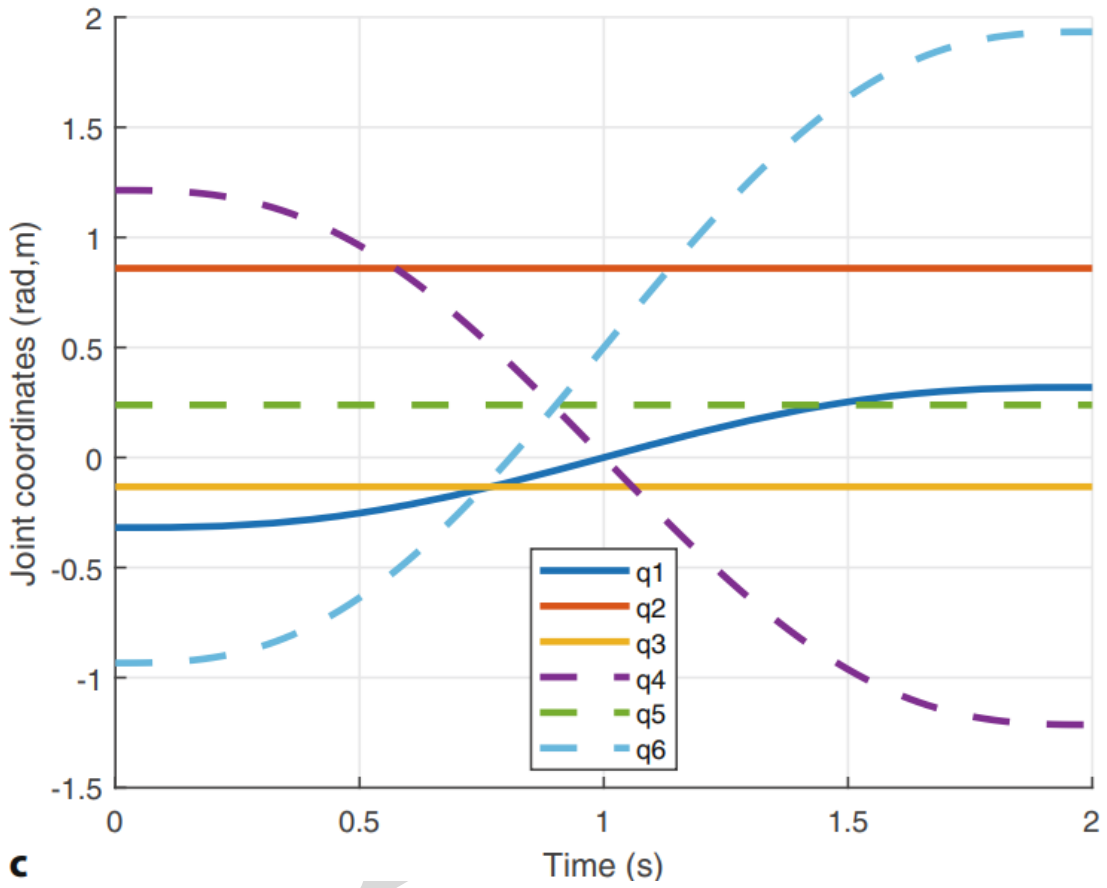
handles the singularity with far less unnecessary joint motion.

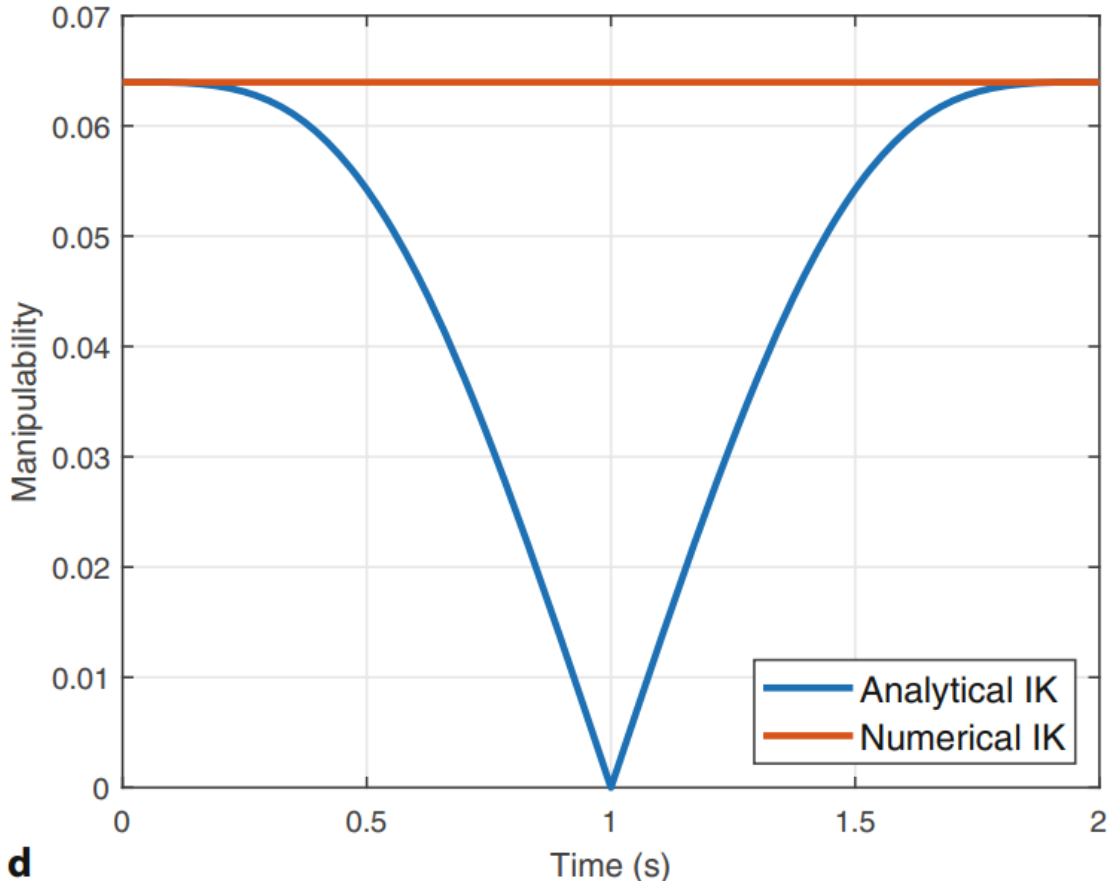
- This is a consequence of the minimum-norm solution which has returned q_4 and q_6 which have the correct sum but the smallest magnitude.



DRAFT

- The joint-space motion between the two poses is immune to this problem since it is does not involve inverse kinematics.
- However, it will not maintain the orientation of the tool for the whole path – only at the two end points.





5.4 Applications

- Our goal is to create a trajectory that will allow a robot to draw a letter.
- Load a preprocessed version of the Hershey font

```
>> load hershey
```

- The path to be drawn is

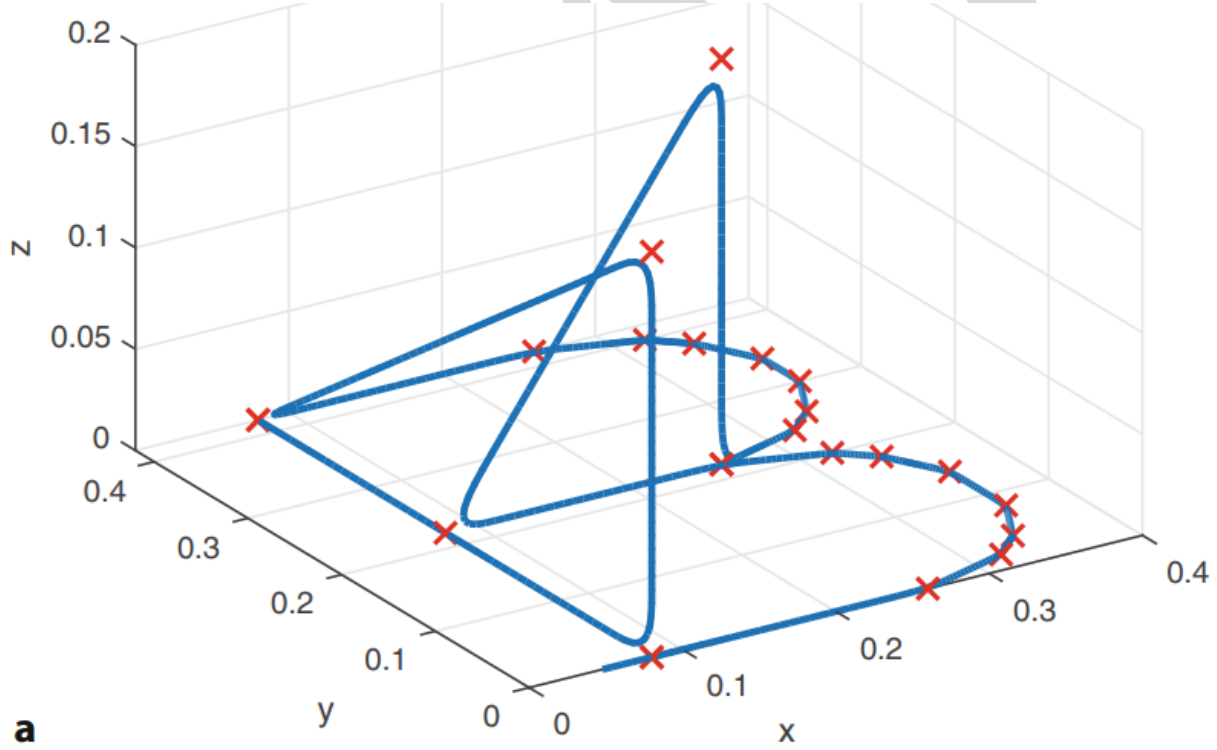
```
>> B.stroke
ans =
  Columns 1 through 9
    0.1600    0.1600       NaN    0.1600    ...
    0.8400         0       NaN    0.8400    ...
  ...
```

where the rows are the x- and y-coordinates respectively, and a column of NaNs indicates the end of a segment – the pen is lifted and placed down again at the beginning of the next segment.

DRAFT

- Find the columns that contain NaNs and replace them with the preceding column but with the z-coordinate set to 0.2 in order to lift the pen off the surface.
- Next, we convert this to a continuous trajectory

```
>> dt = 0.02;  % sample interval
>> traj = mstraj(p(:,2:end)',[0.5 0.5 0.5],[],p(:,1)',dt,0.2);
```



a

- We now have a sequence of 3-dimensional points but for inverse kinematics we require a sequence of end-effector poses.

```
>> Tp = se3(trvec2tform([0.5 0 0.075]))*se3(eye(3),traj)* ...
>>    se3(oa2tform([0 1 0],[0 0 -1]));
```

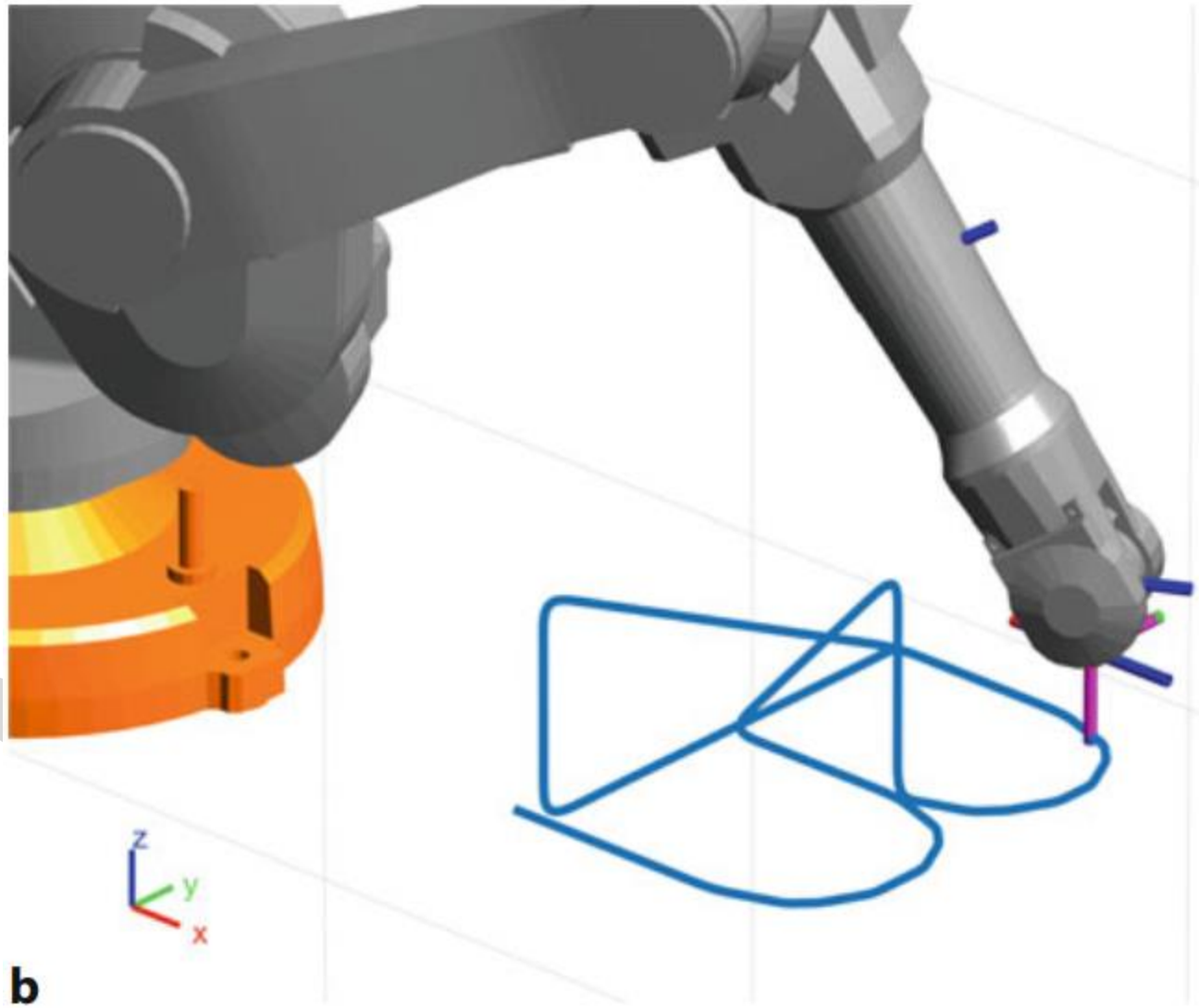
- Now we can apply inverse kinematics

```
>> qj = ikineTraj(abbIKFcn,Tp);
```

to determine the joint coordinates and then animate it

```
>> r = rateControl(1/dt);
>> for i = 1:size(qj,1)
>>    abb.show(qj(i,:),FastUpdate=true,PreservePlot=false);
>>    r.waitfor;
>> end
```

DRAFT



b